B.Tech. BCSE497J - Project-I

MULTI AGENT FRAMEWORK FOR DESKTOP AUTOMATION

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Technology

in

Computer Science and Engineering

by

23BAI0093	**Chethan Krishna M.**
23BCE2115	**Sriram Madala**
23BAI0122	**Gopi Adithya S**

Under the Supervision of

M. Rajasekhara Babu

Professor Grade 2

School of Computer Science and Engineering

(SCOPE)



VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

October 2026

DECLARATION

I hereby declare that the project titled Multi Agent Framework for Desktop Automation, submitted by me to VIT for the award of the degree of Bachelor of Technology in Computer Science and Engineering, is a record of bonafide work carried out by me under the supervision of M. Rajasekhara Babu.

I further declare that the work reported in this project has not been submitted, and will not be submitted, in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Vellore
Date:

Signature of the Candidate
Sriram Madala (23BCE2115)

CERTIFICATE

This is to certify that the project titled Multi Agent Framework for Desktop Automation, submitted by Chethan Krishna M. (23BAI0093), Sriram Madala (23BCE2115) and Gopi Adithya S (23BAI0122), School of Computer Science and Engineering, VIT, for the award of the degree of Bachelor of Technology in Computer Science and Engineering, is a record of bonafide work carried out by them under my supervision during the Fall Semester 2026-2027, in line with the VIT code of academic and research ethics.

The contents of this report have not been submitted, and will not be submitted, in part or in full, for the award of any other degree or diploma in this institute or any other institute or university. The project meets the requirements and regulations of the University and, in my opinion, the standard needed for submission.

Place: Vellore
Date:

Signature of the Guide
M. Rajasekhara Babu
Professor Grade 2, SCOPE

Signature of the Examiner(s)

Dr. Bhoominathan P
Head of the Department
School of Computer Science and Engineering (SCOPE)

ACKNOWLEDGEMENTS

I thank the management of Vellore Institute of Technology (VIT) for the opportunity to carry out this project and for the facilities I used during it.

I thank Dr. Jaisankar N, Dean of the School of Computer Science and Engineering (SCOPE), and Dr. Bhoominathan P, Head of the Department, for their support during the semester.

I am grateful to my guide, M. Rajasekhara Babu, Professor Grade 2, SCOPE, for the guidance and feedback I received throughout the project.

Sriram Madala

TABLE OF CONTENTS

Abstract **xiii**

1. INTRODUCTION **1**

1.1 Background 1

1.2 Motivations 2

1.3 Scope of the Project 3

2. PROJECT DESCRIPTION AND GOALS **4**

2.1 Literature Review 4

2.1.1 Tool-using language models 4

2.1.2 Computer-use agents 4

2.1.3 Windows-native agents 4

2.1.4 Web agents and benchmarks 5

2.1.5 Robotic Process Automation and assistants 5

2.2 Research Gap 5

2.3 Objectives 5

2.4 Problem Statement 6

2.5 Project Plan 6

3. TECHNICAL SPECIFICATION **8**

3.1 Requirements 8

3.1.1 Functional 8

3.1.2 Non-Functional 8

3.2 Feasibility Study 9

3.2.1 Technical Feasibility 9

3.2.2 Economic Feasibility 10

3.2.3 Social Feasibility 10

3.3 System Specification 10

3.3.1 Hardware Specification 10

3.3.2 Software Specification 10

4. DESIGN APPROACH AND DETAILS **12**

4.1 System Architecture 12

4.2 Design 14

4.2.1 Data Flow Diagram 14

4.2.2 Use Case Diagram 14

4.2.3 Class Diagram 16
4.2.4 Sequence Diagram 17
4.2.5 Data model 17

5. METHODOLOGY AND TESTING **19**
5.1 Module Description 19
5.1.1 Implementation environment 19
5.1.2 Task workspace and agent engines 20
5.1.3 Browser tabs that render off-screen 21
5.1.4 Windows applications through UI Automation 22
5.1.5 Borrowed input that gives way to the user 23
5.1.6 Windowless helpers and system changes 24
5.1.7 Chrome extensions for the agent 25
5.1.8 Connectors through the Model Context Protocol 25
5.1.9 Generative-UI answers, triage and approvals 26
5.1.10 Memory, site memory and routines 28
5.1.11 Security review: /bugbounty 29
5.1.12 Phone app and chat channels 29
5.1.13 3D design in the background 29
5.2 Testing 31

6. PROJECT DEMONSTRATION **34**

7. RESULT AND DISCUSSION **41**
7.1 DEX-Bench results 41
7.1.1 Checker corrections and re-scoring 43
7.2 DEX compared with its engines run alone 44
7.3 Background operation 45
7.4 Tool use in recorded tasks 45
7.5 Non-interference as a requirement 47
7.6 Structure before pixels 47
7.7 Safety outside the model 47
7.8 Limitations and threats to validity 48

8. CONCLUSION **50**
8.1 Summary of contributions 50
8.2 Future work 50

9. REFERENCES **52**

A. SAMPLE CODE **55**

A.1 Telling borrowed input from the user 55
A.2 When input may be borrowed 55
A.3 What the CDP broker refuses 56
A.4 When an action needs approval 57

LIST OF FIGURES

1.1 A real task in progress: DEX reads GitHub Trending in the task's own browser tab while the chat lists its steps . . . 2

2.1 Commits per week from the git history, with the dates of the main milestones . 7

4.1 System architecture . . . 13
4.2 Level 0 context data flow diagram . . . 14
4.3 Level 1 data flow diagram . . . 15
4.4 Use case diagram . . . 15
4.5 Class diagram of the core components . . . 16
4.6 Sequence diagram of a background Windows task with borrowed input . . . 17
4.7 Data model, with SQLite and files on the PC and Firestore for the phone link . 18

5.1 Size of the code base by part, from the files tracked in git . . . 20
5.2 The CDP broker gives each task one token, and only that task's tab can be reached 22
5.3 Life cycle of a task's browser in the pool . . . 22
5.4 How DEX picks a background interface, and when it borrows input . . . 24
5.5 Settings > Desktop control, with background app use, per-app rules, the off-limits list and recent changes . . . 25
5.6 The connector Marketplace . . . 26
5.7 Signing in to a hosted MCP server with discovery, dynamic registration and PKCE . . . 27
5.8 Settings > About you, with optional questions that seed DEX's memory and an import from another assistant . . . 28
5.9 DEX for Android, rendered from the app's code with sample data: home, answers as cards, and an approval . . . 30
5.10 DEX for Android, rendered with sample data: a live task, routines, and a routine shown as a graph . . . 30
5.11 Settings > Channels, with the phone, WhatsApp, Telegram and advanced options 31
5.12 Settings > Diagnostics, with tools, engines, network and warnings . . . 32

6.1 The hub, with running tasks, tasks waiting for the user and recent tasks . . . 34
6.2 The answer to the GitHub task, today's top repositories as cards with their stars and descriptions . . . 35
6.3 A real task operating Notepad's Save As dialog through the Windows tools, seen in the Desk tab . . . 35

6.4 A real flight search answered through the Kiwi.com connector, as cards with Select and Open buttons 36
6.5 A real task adding an all-day event through the Google Calendar connector . . 36
6.6 A real triage card for unread mail, with urgency across, importance up and a drafted reply for each message 37
6.7 A chart widget built from memory for the prompt “Show me a hexagon graph of my strengths, from what you know about me” 38
6.8 An approval card for a background step, with Deny, Once, This turn, Session and Approve 38
6.9 A real task listing and using the installed Chrome extensions on a Wikipedia page 39
6.10 A Japanese shrine scene modelled and rendered by DEX in a windowless Blender 39
6.11 Screenshots from a real phone: a search for the cheapest flights to Canada, started and answered on the phone (23 tool calls) 40

7.1 Tasks passed by each configuration (Claude Opus 5.5 for the first two, gpt-6-luna for the rest) 41
7.2 Every DEX-Bench task in every run, with its time in seconds and failures marked 42
7.3 Time per task with Claude Opus 5.5, DEX against the Claude Code CLI alone 45
7.4 Tool calls by kind in the 45 recorded tasks outside the benchmark 46
7.5 Tool calls per task in the 45 recorded tasks outside the benchmark 46
7.6 Time per tool call by kind across all recorded tasks, as the median and the 90th percentile 47
7.7 After the user denied opening File Explorer, the model emptied the Recycle Bin through PowerShell 48

LIST OF TABLES

3.1 Functional requirements 8
3.2 Non-functional requirements 9

5.1 DEX's modules and where their code lives 19
5.2 Hardware and software implementation environment 20
5.3 The 19 tools of the Windows control server 23
5.4 The 24 DEX-Bench tasks 32
5.5 The four configurations run on DEX-Bench 33

7.1 DEX-Bench results for each configuration 41
7.2 Tasks passed and failed by category 42
7.3 Why each failed task failed, from the checker's record 43

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
APK	Android Package
ASCII	American Standard Code for Information Interchange
CDP	Chrome DevTools Protocol
CI	Continuous Integration
CIM	Common Information Model
CLI	Command-Line Interface
COM	Component Object Model
CPU	Central Processing Unit
CRX3	Chrome extension package format, version 3
CSRF	Cross-Site Request Forgery
CSV	Comma-Separated Values
DNS	Domain Name System
DOM	Document Object Model
DPI	Dots Per Inch
FR	Functional Requirement
GLB	Binary glTF 3D model file
GPU	Graphics Processing Unit
GUI	Graphical User Interface
HTTP	Hypertext Transfer Protocol
IP	Internet Protocol
JSON	JavaScript Object Notation
LLM	Large Language Model
MCP	Model Context Protocol
NFR	Non-Functional Requirement
OAuth	Open Authorization
OS	Operating System
PC	Personal Computer
PDF	Portable Document Format
PKCE	Proof Key for Code Exchange
RAM	Random-Access Memory
RPA	Robotic Process Automation
SDK	Software Development Kit
TLS	Transport Layer Security
UI	User Interface
UIA	UI Automation (Microsoft)
UTC	Coordinated Universal Time

SYMBOLS AND NOTATIONS

I	a task: the instruction in natural language
S_0	state of the user's computer and accounts when the task starts
S_n	state after the last action
a_1, ..., a_n	the sequence of actions the agent takes
A	the set of tools available to the agent
A_{api}	operating-system and application interfaces
A_{uia}	UI Automation patterns
A_{web}	actions in the agent's own browser tab
A_{mcp}	calls to online services through MCP
A_{input}	borrowed mouse and keyboard input
$G_I(S_n)$	the goal implied by I, true when S_n completes the task
U(t)	true when the user is active or watching something full-screen at time t

ABSTRACT

Computer-use agents built on large language models (LLMs) can operate graphical software, but most of them take over the screen, mouse and keyboard while they work, and they are developed and tested inside virtual machines that belong to the agent. This project built DEX, an open-source multi-agent framework for desktop automation on Windows, with a companion Android app. The user writes a task in plain words on the PC, on the phone or in a chat app, and DEX starts an agent for it, using an existing coding-agent command-line tool (Claude Code, Codex or BrowserCode) as the planner. Several tasks can run side by side, each with its own agent, chat and browser tabs. The agents act through tools that work in the background: tabs in DEX's own browser, Windows applications driven through UI Automation while their windows sit off-screen, and online services reached through the Model Context Protocol (MCP), including 51 hosted connectors.

To avoid taking focus, DEX follows a fixed order. It uses a system or application interface when one exists, then UI Automation patterns on windows it has parked off-screen, and it borrows the mouse and keyboard only as a last resort. Borrowed input carries a private marker and is watched by low-level hooks, so DEX lets go as soon as the user moves. Approvals, off-limit applications and an undo journal are enforced in DEX's own tools, where the model cannot change them. We tested DEX on DEX-Bench, 24 everyday tasks on a real Windows PC, each scored by a script that reads the end state. With Claude Opus 5.5, DEX and the Claude Code CLI alone both passed all 24 tasks, and DEX was slower, with a median of 39 s per task against 25.5 s. With gpt-6-luna, DEX failed 4 tasks in each of two runs and the Codex CLI alone failed 5. Testing also found a gap in the approvals: they cover DEX's own tools but not the engine's shell, and in one recorded task a model emptied the Recycle Bin through PowerShell after the user had denied the matching DEX action.

Keywords - Multi-Agent Systems, Computer-Use Agents, Large Language Models, Desktop Automation, UI Automation, Background Execution, Model Context Protocol, Human-in-the-Loop Approval, Benchmarking

CHAPTER 1

INTRODUCTION

1.1 Background

A lot of everyday computer work is small and repetitive and spread over many programs: looking up the cheapest flight, filling in the same form on a college portal, renaming a folder of downloads, summarising a document or answering a pile of mail. Companies automate this kind of work with Robotic Process Automation (RPA). RPA scripts are written by developers against fixed selectors, so they break when a page or a window changes [1].

Large language models made a more flexible kind of automation possible. A model that reasons, calls a tool, reads the result and then decides on the next step [2], [3] can follow an instruction written in plain words and recover when the screen looks different from what it expected. Since 2024, computer-use agents have operated whole desktops from screenshots with a virtual mouse and keyboard [4], [5], and public benchmarks such as OSWorld [6] and Windows Agent Arena [7] now score these agents on hundreds of real tasks.

Most of these agents assume the computer belongs to them. They are built and scored in virtual machines, they keep the screen and the input devices for the whole task, and they often work from pixels even when a program has a structured interface. Each also tends to live in one place, such as a browser extension, a desktop program or a cloud sandbox.

DEX is a framework for running agents on a computer that someone is using at the same time. It is a Windows desktop application written in Electron, React and TypeScript, with a native Android app written in Kotlin. Each task the user starts gets its own agent: DEX hands the task to a coding agent such as Claude Code [8], which plans the work and calls DEX's tools to do it. Several of these agents can run at once, each with its own chat, browser tabs and logs, and an agent can start sub-agents of its own. DEX can ask the user before a risky step. In Fig. 1.1, a task is reading GitHub's Trending page in its own tab while its chat lists the steps.

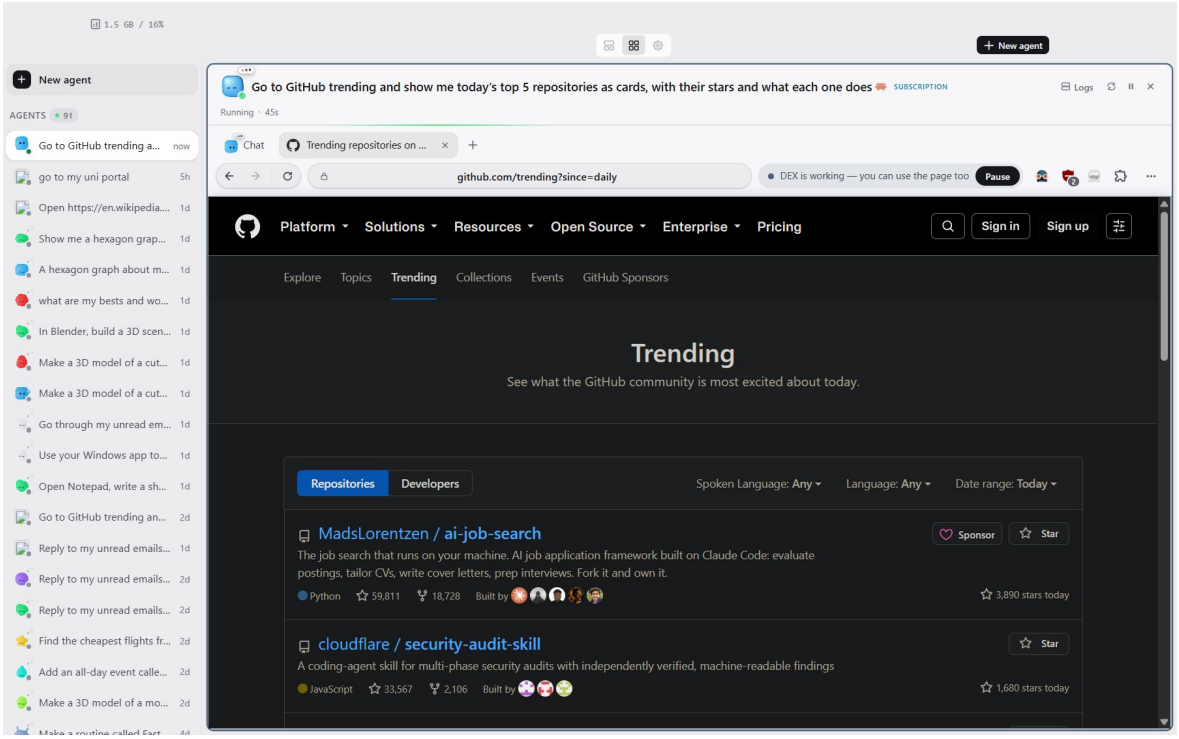


Fig. 1.1 A real task in progress: DEX reads GitHub Trending in the task's own browser tab while the chat lists its steps

1.2 Motivations

The first motivation was the user's time. An agent that takes over a laptop for ten minutes saves its owner nothing if they have to sit and watch. We wanted DEX to keep working while the user types, browses or watches a video, and this requirement decided most of the design.

The second was speed and reliability. In a screenshot loop, every step sends an image to a model, which then has to work out what is on the screen. Windows already exposes most programs through UI Automation (UIA) [9], and many settings and online services can be reached through system commands and APIs. An agent that tries these first and falls back to pixels only when it must should need fewer and faster steps. DEX's task history agrees: across all recorded tasks, the median file operation took 0.07 s and the median UI Automation call 0.22 s, while the median browser call took 0.99 s (Section 7.4).

The third was trust. In August 2026, while testing an early prototype, we found that bringing a window to the front on Windows could deliver a synthetic click to whatever button was under the pointer. In the prototype, that click approved a confirmation card nobody had touched. A timeout could not tell the injected click from a real one, because it arrived anywhere from a few milliseconds to more than 1.2 seconds later. Pointer movement could: a person has to move onto a button before pressing it, while the injected click lands wherever the pointer already is. After this we put safety checks in code that the model cannot change, and made sure injected input could not satisfy them.

People are also not always at their desks. A task can be started from a phone, approved from a notification or sent to a chat bot. Of the 45 recorded tasks outside our benchmark, 17 were started from the Android app.

1.3 Scope of the Project

The project covers the design, implementation and evaluation of an installable framework for Windows 10 and 11 with an Android companion app. The work in scope is:

- a desktop app that runs several tasks side by side, each with its own agent, chat, browser tabs, documents and logs, using Claude Code, Codex or BrowserCode as the planner;
- background execution, meaning browser tabs that render off-screen, Windows applications driven without taking focus, borrowed input that gives way to the user, and windowless helpers such as Blender;
- connections to online services through built-in Google, Microsoft, GitHub, Slack, Reddit, Hugging Face and Discord accounts and 51 hosted MCP connectors, plus Chrome extensions in the agent's browser and control from Android, Telegram, WhatsApp and Discord;
- features built on these, including user memory, site memory, scheduled routines, answers as interactive cards, a triage card for replying to messages, 3D design and an observation-only website security review (/bugbounty);
- safety through approval categories decided outside the engine, off-limit applications, secret handling and an undo journal;
- evaluation with DEX-Bench, a 24-task benchmark with scripted end-state checks, run on DEX with two engines and on each engine alone, together with an analysis of DEX's recorded task history and its unit tests.

We did not train or fine-tune a model; DEX drives existing models through their official command-line tools. macOS and Linux are out of scope, as is any offensive security capability, since the security command only observes.

CHAPTER 2

PROJECT DESCRIPTION AND GOALS

2.1 Literature Review

2.1.1 Tool-using language models

ReAct [2] showed that a model which alternates reasoning with actions can solve multi-step problems by calling tools and reading their output. Toolformer [3] showed that a model can learn when an API call helps. Coding agents such as Claude Code [8] and Codex [10] run this loop on software projects, reading files, running commands and editing code until a goal is met. Anthropic introduced the Model Context Protocol (MCP) [11] in November 2024 as a standard way for agents to find and call tools. One MCP server, for Gmail for example, works with any agent that speaks the protocol, and DEX depends on this.

2.1.2 Computer-use agents

Anthropic's computer use [4] and OpenAI's Computer-Using Agent [5] let a model operate a whole desktop from screenshots with virtual mouse and keyboard actions. Agent S [12] added hierarchical planning and memory of past experience. Agent S2 [13] divided the work between a general planner and specialist grounding models, and Agent S3 [14] runs several attempts at a task and picks the best. All of them are developed and evaluated in virtual machines that the agent controls.

2.1.3 Windows-native agents

Microsoft's UFO [15] was one of the first agents to read the Windows UI Automation tree in addition to pixels. UFO2 [16] grew into a "desktop AgentOS" that mixes GUI actions with native application APIs, and it runs tasks in a separate Picture-in-Picture desktop so that the user is left alone. UFO2 is the closest prior work to DEX, and our first prototype, in June 2026, used it as a driver. DEX shares its preference for structured interfaces and stays out of the user's way by other means. It works inside the user's own session, parks the windows it opens off-screen, acts on them through accessibility patterns and borrows real input only when the user is idle.

2.1.4 Web agents and benchmarks

WebArena [17] and Mind2Web [18] built realistic web tasks. Browser Use [19] became a popular open-source library for driving Chromium through the Chrome DevTools Protocol (CDP) [20], and DEX's browser tooling descends from it. GAIA [21] poses assistant questions that need browsing, files and reasoning. OSWorld [6] has 369 tasks in an Ubuntu virtual machine, and Windows Agent Arena [7] has 154 tasks in a Windows 11 virtual machine. Both score a task by checking the final state with a script instead of trusting the agent's report, and DEX-Bench uses the same method.

2.1.5 Robotic Process Automation and assistants

RPA tools record or script workflows against UI selectors [1]. They are predictable, but they break easily and someone has to build them. Voice assistants are easy to start and reach only a few programs. DEX sits between the two: the user starts it in plain words, as with an assistant, it reaches about as many programs as RPA, and it is meant to be safe on a personal machine.

2.2 Research Gap

Four gaps matter to someone who wants an agent on their own PC.

1. Published computer-use agents assume they own the screen. UFO2 avoids interference by using a separate desktop session, and benchmark reports do not say whether a person could have kept working during a run.
2. Most agents act through pixels at every step. System interfaces, application APIs and accessibility patterns are more reliable, but few agents combine them with a browser and online-service connectors.
3. Approval prompts usually live in the agent's own loop or prompt, where a synthetic click, a mistaken tool call or another tool that reaches the same result can get past them.
4. Agents are usually developer tools that need API keys and configuration. They rarely have a phone app, chat channels or one-click sign-in to services.

DEX addresses all four. We measure task success with end-state checks. Non-interference is part of the design, but as Chapter 7 explains, we have not yet measured it directly.

2.3 Objectives

The objectives of the project were:

1. to design a task model in which every task has its own agent and workspace (chat, browser tabs, documents and logs), several tasks run in parallel, and the engine behind each agent can be swapped;
2. to build background execution that does not take the user's screen, focus or input while they are active;
3. to connect agents to the user's online services through MCP with one-click OAuth sign-in, and to let them use Chrome extensions;
4. to enforce safety outside the model with approval categories, off-limit applications, secret handling and undo;
5. to make DEX usable from a phone and from chat apps, with answers shown as interactive cards;
6. to evaluate DEX on tasks checked by scripts, against the same engines run without DEX.

2.4 Problem Statement

Let a task be an instruction I in natural language, and let S_0 be the state of the user's computer and accounts when the task starts. The agent chooses a sequence of actions a_1, ..., a_n from a tool set A = $A_{api} \cup A_{uia} \cup A_{web} \cup A_{mcp} \cup A_{input}$, whose parts are, in order, operating-system and application interfaces, UI Automation patterns, actions in the agent's own browser tab, calls to online services, and borrowed mouse and keyboard input. The final state S_n must satisfy the goal $G_I(S_n)$ implied by I. Two constraints separate this problem from the usual benchmark setting.

- Non-interference. Let U(t) be true when the user has produced input in the last moment or is watching something full-screen. While U(t) holds, the agent must not change the foreground window, move the cursor or inject input. Actions from A_{input} are allowed only while U(t) is false, and they stop at the first input that did not come from the agent.
- Consent. An action with consequences, such as sending, deleting, paying or changing a system setting, should follow the user's approval of exactly that action, and neither the agent nor synthetic input may be able to give that approval.

Within these constraints the agent should finish in as few steps and as little time as it can, and it should prefer the earlier members of A, which are faster and less ambiguous than pixels.

2.5 Project Plan

The git history records 473 commits between 3 June and 20 October 2026 (Fig. 2.1). In June (163 commits) we built a first prototype on a fork of an existing agent framework,

with UFO2 driving Windows. Little was committed in July. In late August we built a Flutter desktop bar with confirmation cards, which is where we measured the injected-click problem. The Electron desktop app entered the repository on 10 September, and the old Flutter client and its Node and Python back end were removed on 19 September. September (135 commits) and October (140 commits up to 20 October) went into the task workspace, the browser broker, the connectors, the Android app, background Windows control and the features built on them. We tagged v1.0.0 on 7 October, v2.0.0 on 14 October, v2.1.0 on 18 October, and v2.1.1 and v2.2.0 on 19 October. DEX-Bench was built and run on 20 and 21 October.

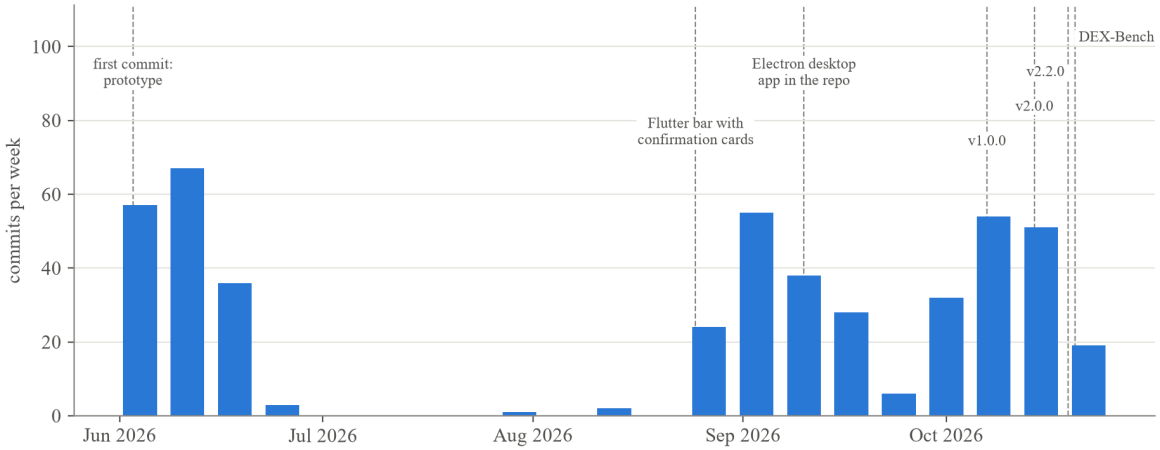


Fig. 2.1 Commits per week from the git history, with the dates of the main milestones

CHAPTER 3

TECHNICAL SPECIFICATION

3.1 Requirements

The requirements come from requests recorded during development in the repository's hand-over file, from failures seen in live use, and from the safety model written before Windows control was built.

3.1.1 Functional

Table 3.1 Functional requirements

ID	Area	The system shall
FR1	Task intake	accept a task in plain words from the desktop hub, the Android app, Telegram, WhatsApp or Discord, and start it with a chosen engine and model
FR2	Parallel tasks	run several tasks at once, each with its own agent, chat, browser tabs, documents and logs, and let the user watch, steer, pause, take over or cancel any of them
FR3	Web automation	browse, fill forms, read pages and download files in browser tabs the user can also use, including pages that need Chrome extensions
FR4	Windows automation	open and operate installed Windows applications, read and change supported system settings, and capture the window being worked on, without taking focus while the user is active
FR5	Connected services	connect Google, Microsoft 365, GitHub, Slack, Discord and the hosted connectors with one sign-in each, and give the agent their tools through MCP
FR6	Interactive answers	show questions and results as widgets (pickers, cards with buttons, charts, a triage plot) on the desktop, the phone and chat apps, and treat a tap as the user's reply
FR7	Approvals	ask before a consequential step, say what kind of step it is, and accept the answer from the desktop or the phone
FR8	Memory and routines	remember facts the user allows, notes about sites and named routines, and run routines on a schedule
FR9	Security review	offer an observation-only website review for targets the user is authorised to test
FR10	3D design	model and render 3D scenes in Blender in the background and show them on the PC and the phone

3.1.2 Non-Functional

Table 3.2 Non-functional requirements

ID	Area	Requirement
NFR1	Non-interference	No task takes the foreground, the cursor or the keyboard while the user is active, and borrowed input stops at the first user movement.
NFR2	Performance	The interface stays responsive with ten or more tasks open. Idle tasks are frozen, and the least recently used idle browser is freed when the pool of ten is full.
NFR3	Security	No remote-debugging port is open by default. Secrets stay in the operating system's credential store, stored passwords are never returned to the agent, DEX's input tools never type into password fields, and the agent cannot press DEX's own approval cards.
NFR4	Reliability	Every task is stored in SQLite and survives a restart.
NFR5	Usability	Steps and titles appear in plain words instead of raw commands, and one-click sign-in replaces tokens and file paths.
NFR6	Privacy	Memory, site notes, tasks and captures stay on the PC, and the phone link uses the user's own Firebase project.
NFR7	Maintainability	TypeScript in strict mode, unit tests for each subsystem, an MCP-based tool boundary and engines behind a common adapter interface.
NFR8	Licensing	Open source under Apache-2.0, with a licence scan before each release.

3.2 Feasibility Study

3.2.1 Technical Feasibility

Every building block DEX needs already exists and is documented. Electron's WebContentsView [22] and CDP [20] cover the browser, Windows UI Automation [9] and SendInput [23] cover applications, MCP [11] covers services, and OAuth 2.0 with PKCE and dynamic client registration [24], [25], [26], [27], [28] covers sign-in. The Claude Code and Codex command-line tools run as subprocesses with a supplied prompt and tool configuration and stream their progress as JSON events, so DEX needs no model of its own.

The Windows host is C# code that Windows PowerShell 5.1 compiles at run time with the .NET Framework shipped in Windows, so there is no native Node add-on to build. Our design notes record that it compiles and loads in about 0.2 s. The main technical risks were model usage limits, quirks in the user interfaces of individual applications, and the Windows rules for bringing a window to the front. The design deals with them by waiting and retrying, by preferring structured interfaces, and by parking windows instead of activating them.

3.2.2 Economic Feasibility

All the libraries are open source, and development needed one laptop. DEX uses the user's existing model subscription through the official command-line tools and has no server of its own: the phone link runs on the user's own Firebase project, and chat bots are polled from the PC. Nine of the 51 hosted connectors need no sign-in. GitHub Actions builds each release from a version tag, and the Android app is on the Play Store internal-testing track (Section 5.1.1).

3.2.3 Social Feasibility

The user can watch every step, pause a task, take over or hand back, and system changes made through DEX's tools can be undone. DEX captures only the window it is working in, never the whole screen. Password managers, Windows security and sign-in screens, terminals and other AI agents are off-limits to its Windows tools.

The /bugbounty command starts with an authorisation check: the target must belong to the user, be in a published bug-bounty scope or be explicitly permitted. After that it only observes, with no exploitation, injection, authentication bypass, brute force or denial of service. Memory is generic and asks optional "about you" questions instead of assuming who the user is.

3.3 System Specification

3.3.1 Hardware Specification

DEX was developed and tested on a laptop with an AMD Ryzen 9 5980HX (8 cores, 16 logical processors), 16 GB of RAM, an AMD Radeon RX 6800M with integrated Radeon graphics and a 2560×1440 display at 125% scaling, running Windows 11 Home (build 26300). A OnePlus Android phone was used over USB debugging and through the Firebase bridge. Users need Windows 10 or 11 on x64 and, for the companion app, a phone with Android 8.0 or later (API level 26).

3.3.2 Software Specification

The desktop app runs on Electron 41.2.1 with React 19.2.5, TypeScript 5.6.3 and Vite 6.4.2, and it is packaged with Electron Forge. Tasks are stored in SQLite through better-sqlite3, and secrets in the Windows Credential Manager through keytar. Windows control runs in a Windows PowerShell 5.1 host with C# code (DexDesk.cs) that uses UI Automation through COM interop and Win32 functions such as SetWindowPos, PrintWindow, SendInput and low-level hooks.

The engines are the Claude Code CLI (with Claude Opus 5.5, Sonnet 5 and Haiku 4.5),

the OpenAI Codex CLI (gpt-6-luna in the benchmark) and BrowserCode, a command-line agent built on OpenCode. The Android app is written in Kotlin with Jetpack Compose and Material 3 Expressive [29], uses Firebase Authentication and Cloud Firestore, and targets SDK 36 with a minimum of 26. 3D work uses Blender [30] in background mode with an MCP server, and text-to-3D runs through FLUX.1-schnell [31] and Hunyuan3D [32] on Hugging Face ZeroGPU Spaces. Testing uses Vitest 4.1.4, ESLint, TypeScript checks, Robolectric screenshot tests for Android and the DEX-Bench checkers.

CHAPTER 4

DESIGN APPROACH AND DETAILS

4.1 System Architecture

DEX has five layers (Fig. 4.1).

- The user layer is the desktop hub (a React renderer), the Android app and chat apps.
- The Electron main process holds all state and policy: the session manager and task store, approvals, the channel router and phone bridge, workflows, memory, connector sign-in, the MCP catalog and the CDP broker. The renderer reaches it only through a narrow preload bridge.
- In the engine layer, the main process starts one agent per task as a subprocess. The agent is a coding-agent command-line tool given DEX's harness, which is an AGENTS.md guide, a set of dex-* command-line tools and a generated MCP configuration. Several agents run at once, and Claude Code agents can start sub-agents, which DEX tracks separately.
- The tool layer is where agents act, and all of it works in the background: the task's browser tabs, the Windows control server, the built-in MCP servers and the hosted connectors.
- Outside DEX are the websites, Windows applications and cloud services that the tools reach.

Two design rules come up throughout. The first is that the engine is not trusted. It plans well but makes mistakes, so the checks DEX enforces sit in the main process or in the tool servers, out of the engine's reach. Claude Code, for example, is started with a generated MCP configuration and the --strict-mcp-config flag, so it sees exactly the servers DEX lists and none from the user's own set-up. The engines' own permission prompts are switched off, though, which leaves commands that an engine runs in its own shell outside DEX's checks (Section 7.7). The second rule is to prefer structure: an MCP call before a web page, and an API or accessibility pattern before a click.

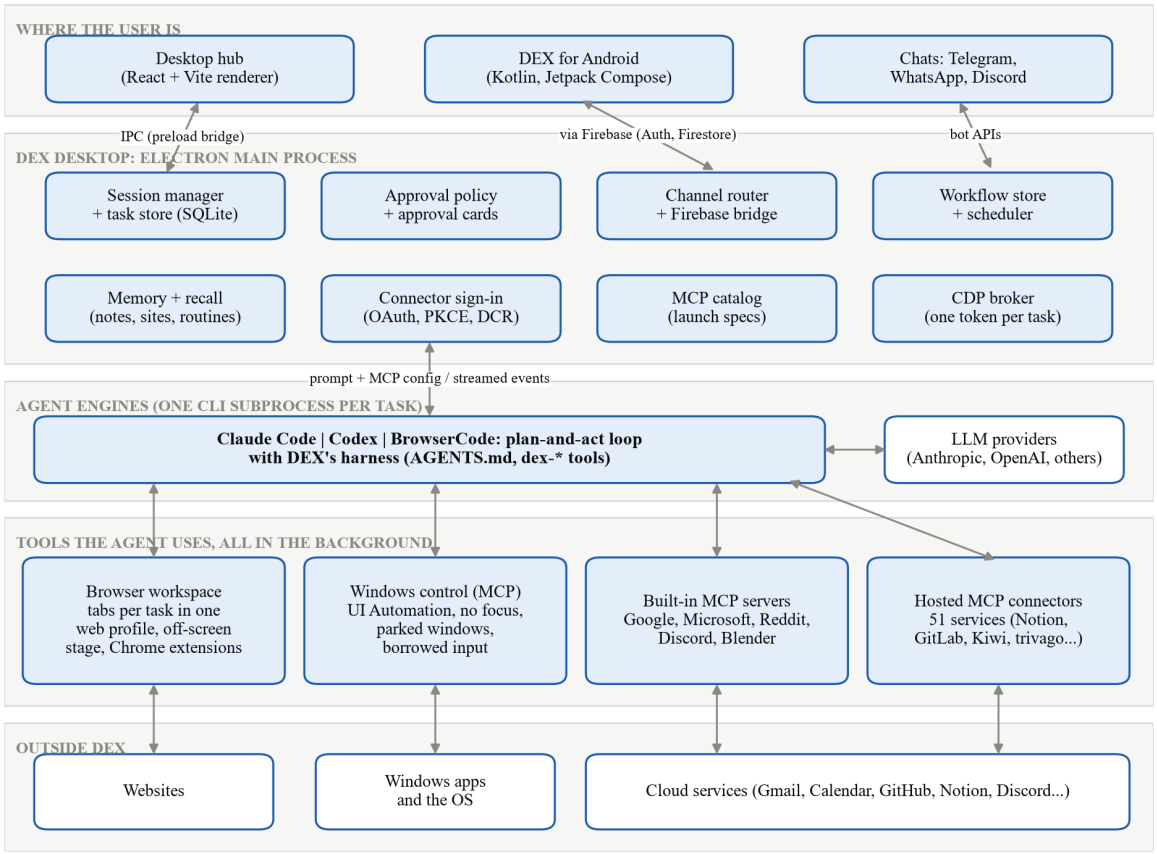


Fig. 4.1 System architecture

4.2 Design

4.2.1 Data Flow Diagram

At the context level (Fig. 4.2), DEX exchanges tasks, answers, approvals and notifications with the user on the desktop, on the phone or in chats. It sends prompts to the LLM provider and gets plans back, acts on web pages, sends UI actions to Windows applications and receives captures, and calls connected services.

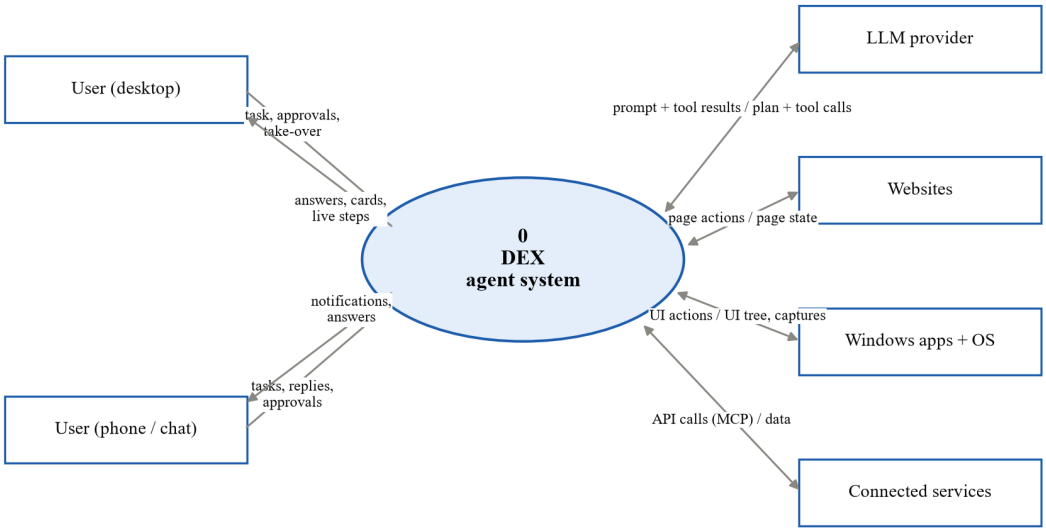


Fig. 4.2 Level 0 context data flow diagram

Level 1 (Fig. 4.3) splits the system into six processes. Process 1.0 receives a task from any surface, and 2.0 builds its context from the relevant memory notes, any site note for the pages involved, the installed extensions and the connectors to attach. Process 3.0 runs the engine's plan-and-act loop and sends tool calls to 4.0, which executes them in the browser, in Windows or through MCP. A consequential step first passes 5.0, the approval gate, and 6.0 turns the outcome into transcript events, cards and widgets. Four stores support them: the task database (D1), memory and site notes (D2), the credential store (D3), and the workflows file with the undo journal (D4).

4.2.2 Use Case Diagram

The primary actor is the user at the PC, who can use every feature (Fig. 4.4). The same person on a phone or in a chat can start, watch and steer tasks, approve steps, run routines and answer triage cards. The LLM provider and the connected services are secondary actors.

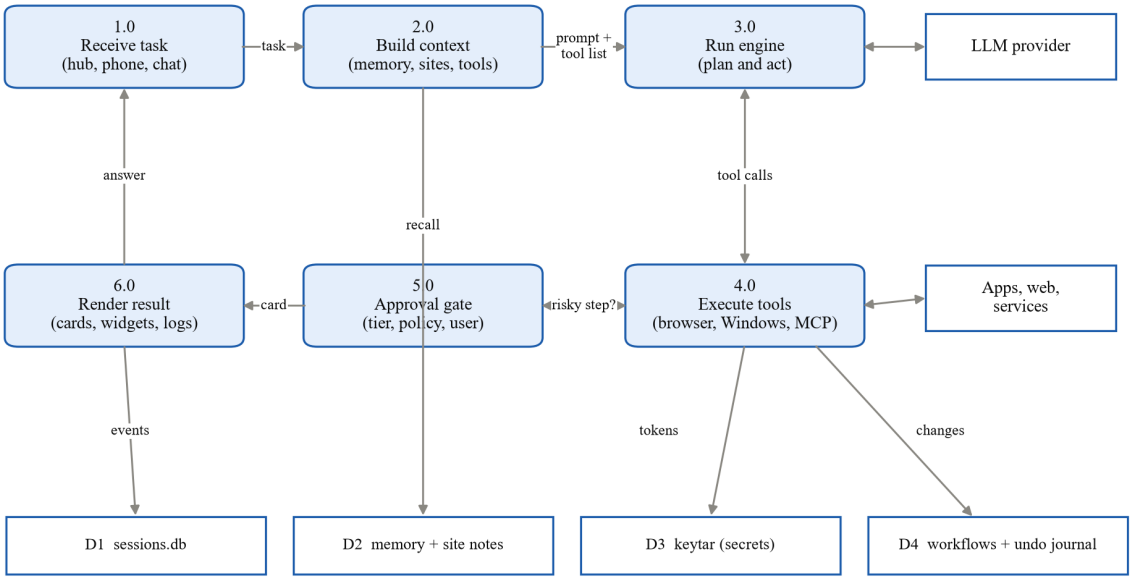


Fig. 4.3 Level 1 data flow diagram

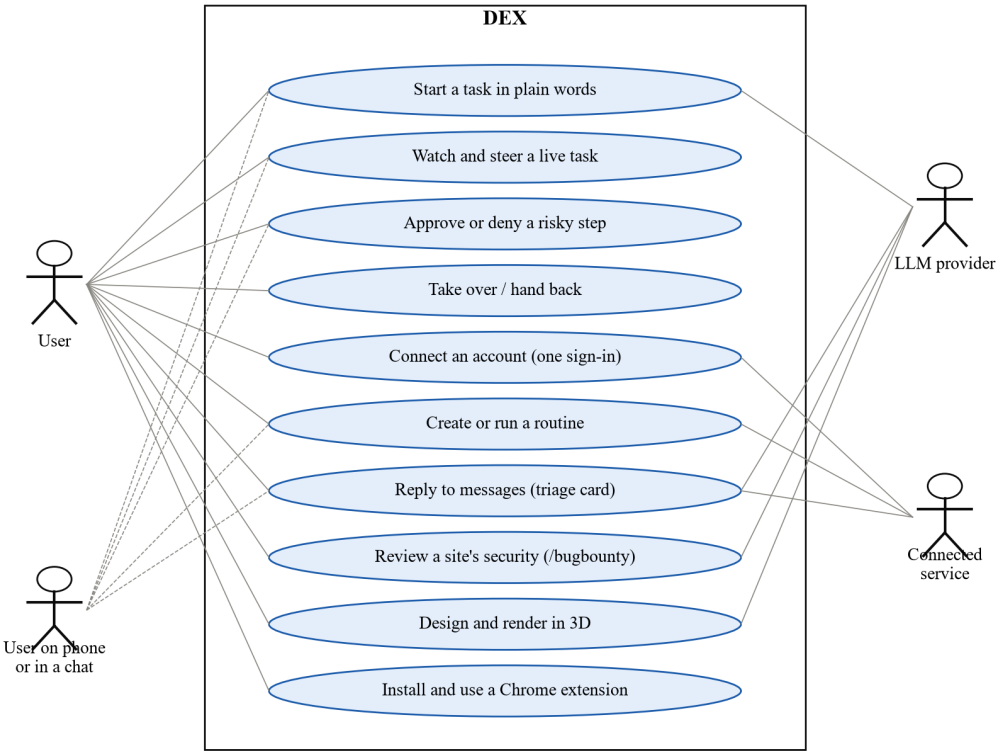


Fig. 4.4 Use case diagram

4.2.3 Class Diagram

The core classes appear in Fig. 4.5 under the names used in the code.

- SessionManager creates, starts and cancels sessions and stores their events.
- Every engine implements the EngineAdapter interface (buildSpawnArgs, buildEnv, getStdinPayload, parseLine), so Claude Code, Codex and BrowserCode can replace one another.
- BrowserPool holds the tabs of up to ten tasks and frees the least recently used idle one when it is full. CdpBroker gives each task an endpoint that reaches only that task's tab, and ExtensionHost lets the agent use extensions but refuses password managers and wallets.
- The MCP catalog knows how to launch each built-in server, including the Windows control server, whose host is the C# class DexDesk.
- ChannelRouter passes Telegram, WhatsApp and Discord messages into the same task flow.

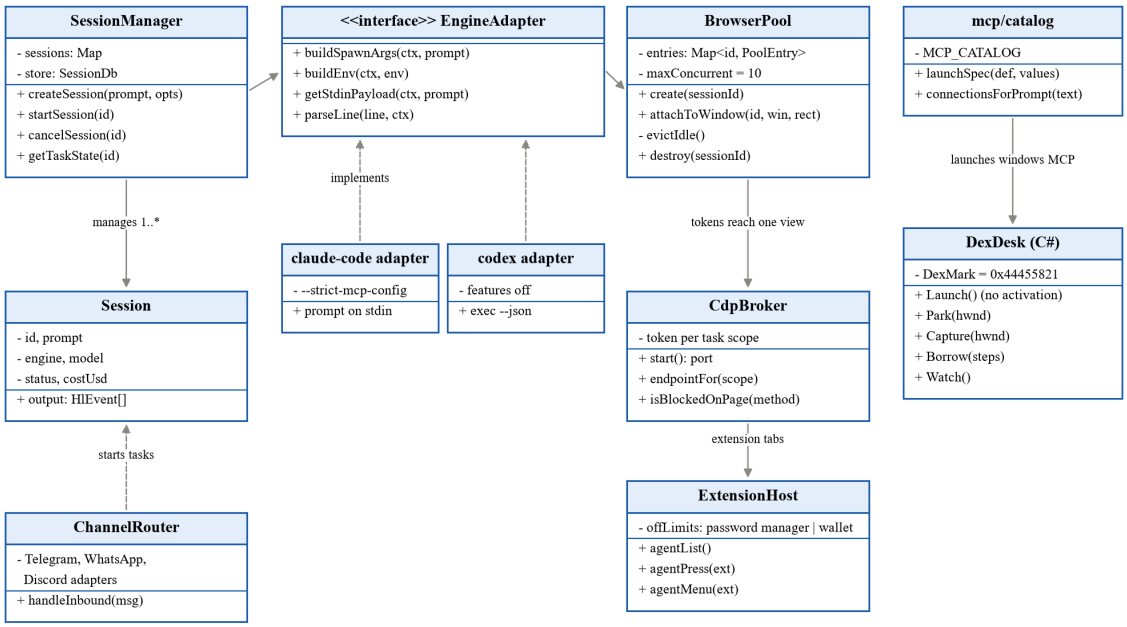


Fig. 4.5 Class diagram of the core components

4.2.4 Sequence Diagram

Fig. 4.6 follows the task “open Notepad, type a short poem and save it” from start to finish. The engine asks the Windows server to launch Notepad, which starts without activation and is parked off-screen. The text goes in through the UIA Value pattern, which needs no focus. Saving needs a real keystroke, so the server borrows input. If the user moves the mouse during the borrow, a low-level hook sees input without DEX’s marker. The server then stops, restores the user’s window, foreground and cursor, and tells the engine that the user is active. Once the user is idle again, the save finishes and the window is parked again.

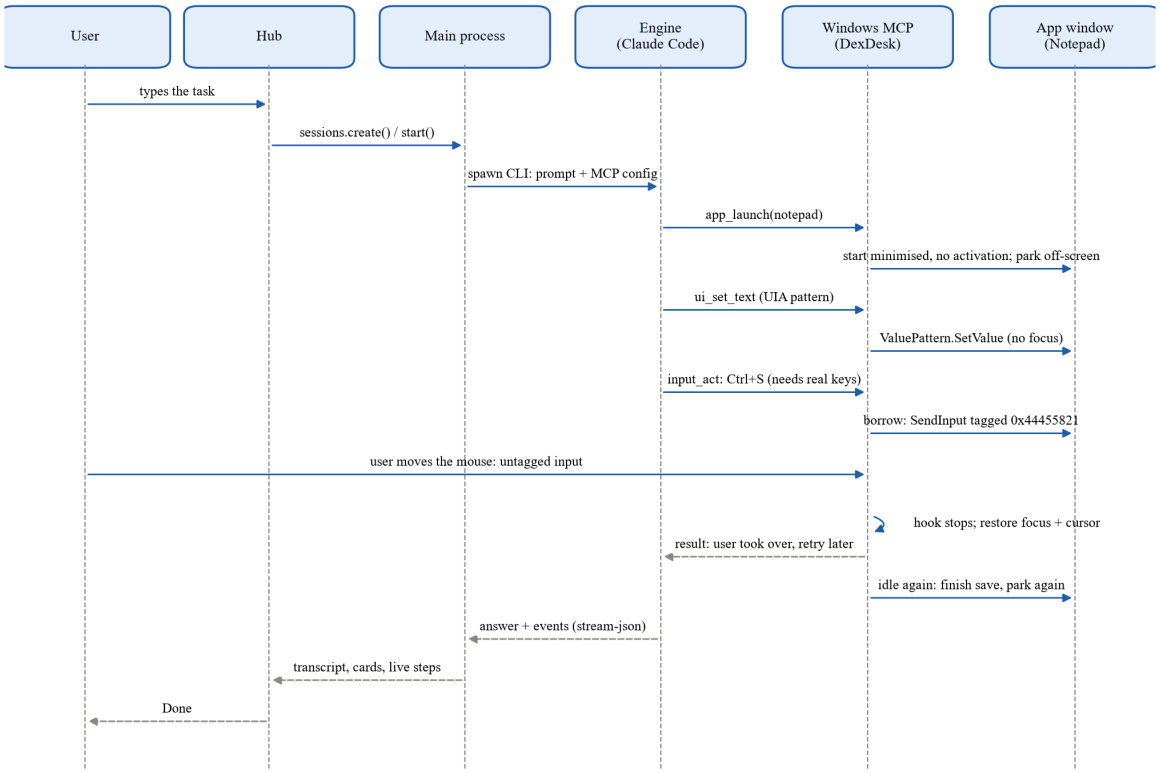


Fig. 4.6 Sequence diagram of a background Windows task with borrowed input

4.2.5 Data model

On the PC, sessions.db holds every task and its event stream, with tables for task state and workspace tabs (Fig. 4.7). Memory, site aliases, site notes, routines and the undo journal are files in the application data folder, and tokens and site logins are in the Windows Credential Manager, which DEX reaches through keytar. In the cloud, the user’s own Firestore project holds devices, task transcripts and phone commands under users/{uid}, and its security rules let a signed-in user reach only their own documents.

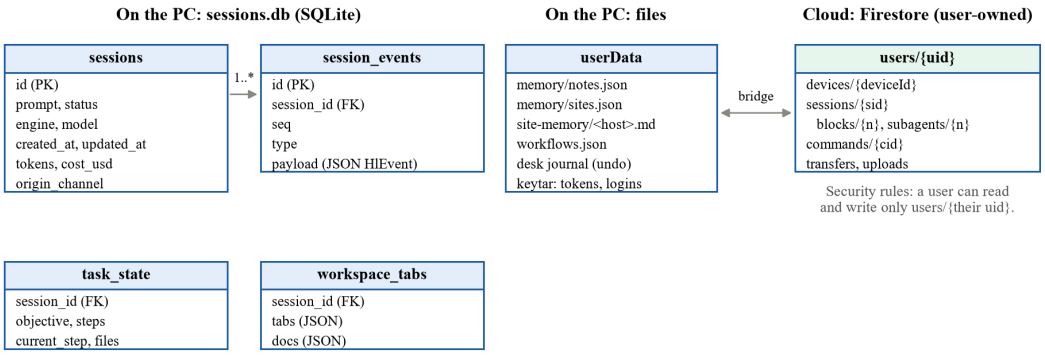


Fig. 4.7 Data model, with SQLite and files on the PC and Firestore for the phone link

CHAPTER 5

METHODOLOGY AND TESTING

5.1 Module Description

DEX is split into the modules in Table 5.1. Paths are relative to desktop/app in the repository, except for the Android app, which has its own folder at the top level.

Table 5.1 DEX's modules and where their code lives

Module	What it does	Main source
Task workspace and engines	Creates tasks, starts one engine command-line tool per task and parses its event stream	src/main/sessions/, src/main/hl/engines/
Browser workspace	Task tabs in a shared web profile, the off-screen stage and the CDP broker	src/main/sessions/BrowserPool.ts, src/main/cdpBroker.ts, src/main/workspace/
Windows control	UI Automation in the background, parked windows, borrowed input and system actions	mcp-servers/windows/
Chrome extensions	Installs and runs store extensions and exposes them to the agent	src/main/extensions/
Connectors	Built-in accounts and hosted MCP servers with OAuth sign-in	src/main/accounts/, src/main/mcp/, src/shared/connectorCatalog.ts
Answers and approvals	Widgets, the triage card and the approval policy	src/shared/widgets.ts, src/shared/genui.ts, src/shared/triage.ts, src/main/approvals/
Memory and routines	User memory, site memory, routines and the scheduler	src/main/memory/, src/main/workflows/
Security review	The /bugbounty and /scrape commands	src/shared/slashCommands.ts
Phone and channels	Android app, Firebase bridge, Telegram, WhatsApp and Discord	android/, src/main/firebase/, src/main/channels/
3D design	Windowless Blender host and text-to-3D	mcp-servers/blender/, src/main/threed/
Desktop interface	The hub, task panes and Settings	src/renderer/

5.1.1 Implementation environment

Table 5.2 lists the environment used to build and test DEX, and Fig. 5.1 breaks the code base down by part. Counting every line in the tracked source files, comments and blank lines included, the code base has 159,556 lines: 116,612 in the desktop app and its MCP servers, 27,287 in the desktop tests and 15,657 in the Android app.

Table 5.2 Hardware and software implementation environment

Component	Specification
CPU, RAM and GPU	AMD Ryzen 9 5980HX (16 logical processors), 16 GB, AMD Radeon RX 6800M
Operating system	Windows 11 Home (build 26300)
Desktop framework	Electron 41.2.1, React 19.2.5, TypeScript 5.6.3, Vite 6.4.2
Agent engines	Claude Code CLI 2.1.280, Codex CLI 0.159.3, BrowserCode
Windows host	Windows PowerShell 5.1 with C# (UI Automation, Win32)
Storage	SQLite (tasks), JSON files (memory, workflows), keytar (secrets)
Mobile	Kotlin, Jetpack Compose, Material 3 Expressive, Firebase
Testing	Vitest 4.1.4, ESLint, Robolectric, DEX-Bench
Version control and CI	Git and GitHub, GitHub Actions

The engine versions are those installed on the development PC when this report was written. The Codex runs in Chapter 7 used Codex CLI 0.159.3, according to Codex's own session logs.

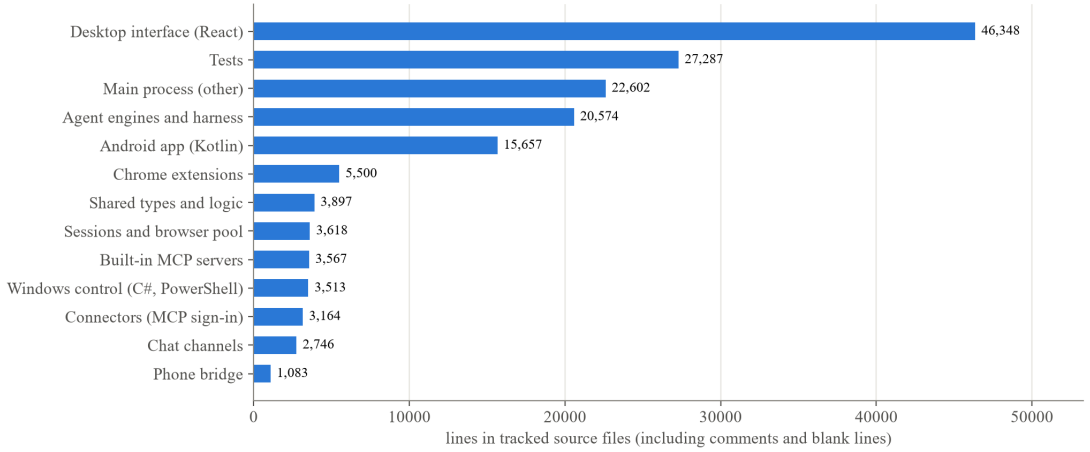


Fig. 5.1 Size of the code base by part, from the files tracked in git

GitHub Actions builds each release from its version tag. The latest release, v2.2.0, was published on 19 October 2026 with a Windows installer (dex-setup.exe) and version 2.2.1 of the Android app as an APK, both on the GitHub releases page [33]: https://github.com/Chethan616/Dex/releases/latest. The Android app is also on the Google Play internal-testing track [34], which only testers added by the developer can open: https://play.google.com/apps/internaltest/4701152187528681647.

5.1.2 Task workspace and agent engines

A task is created with a prompt, an engine and a model, and it is stored in SQLite together with every event it produces. When the task starts, the main process prepares the engine's environment:

- a wrapped prompt that carries DEX's rules, the matching memory and the installed extensions;

- a generated MCP configuration with exactly the servers this task needs;
- the endpoint and token for the task's browser;
- the folder for site notes.

DEX then starts the engine's command-line tool and parses its JSON event stream into DEX's own event types: messages, tool steps, widgets, approvals and files. Claude Code receives the prompt on standard input, because on Windows its command shim cuts an argument at a raw newline. DEX switches off the parts of each engine that duplicate its own features. For Claude Code these are the schedulers (CronCreate, RemoteTrigger and related tools) and slash commands, and for Codex they are its automations and its browser and computer-use features. Routines and browsing therefore always go through DEX's local implementations, where the user can see them.

Within a task, a Claude Code agent can start sub-agents with its Task (or Agent) tool. DEX reads each sub-agent's start, steps and result from the event stream, keeps them out of the main agent's step list, and shows them in a Subagents pane on the PC and on the phone, where each one's prompt, steps and result can be opened. The Codex command-line tool does not report sub-agents in its event stream, so Codex tasks show none.

The hub groups tasks into running, waiting for the user and finished (Fig. 6.1). Opening a task shows a pane with a chat tab, browser and document tabs, a Desk tab for Windows work and a Logs view, and steps are labelled in plain words instead of raw commands.

5.1.3 Browser tabs that render off-screen

The browser, Windows and helper modules follow one rule, written down before the Windows code: the user keeps their PC while DEX works. DEX's tools do not bring windows to the front, take keyboard focus or move the cursor. The only exception is borrowed input (Section 5.1.5), which happens only while the user is idle.

Every task has its own tabs, and each tab is an Electron WebContentsView [22]. All task tabs share one persistent web profile, separate from DEX's own windows, so the user's logins work in every task as they would in a normal browser. Tabs that are not on screen are attached to an off-screen "stage" window, where they keep rendering and can be captured. DEX never touches the user's own Chrome.

The agent reaches its tab through a CDP broker (Fig. 5.2). Early versions opened Chromium's remote-debugging port instead, which exposed every DEX window, approval cards included, to any local process, including the agent's own shell. The broker listens on a random loopback port and gives each task a secret token that reaches only that task's tab. It refuses Browser.* methods, Page.close and Page.crash, and every Target.* method except Target.setAutoAttach and Target.getTargetInfo (Appendix A.3).

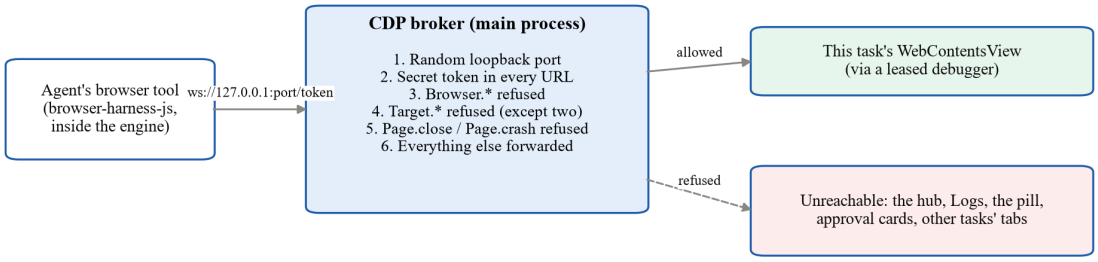


Fig. 5.2 The CDP broker gives each task one token, and only that task's tab can be reached

The user and the agent share the same live tabs. The agent's cursor is drawn inside a closed shadow DOM, where the page cannot see or disturb it, and "take over" hands a tab to the user. The pool holds the tabs of ten tasks at a time. Idle tasks are frozen, and when the pool is full, the browser of the least recently used idle task is freed (Fig. 5.3). This change fixed a "Browser pool full" error we hit after ten benchmark tasks.

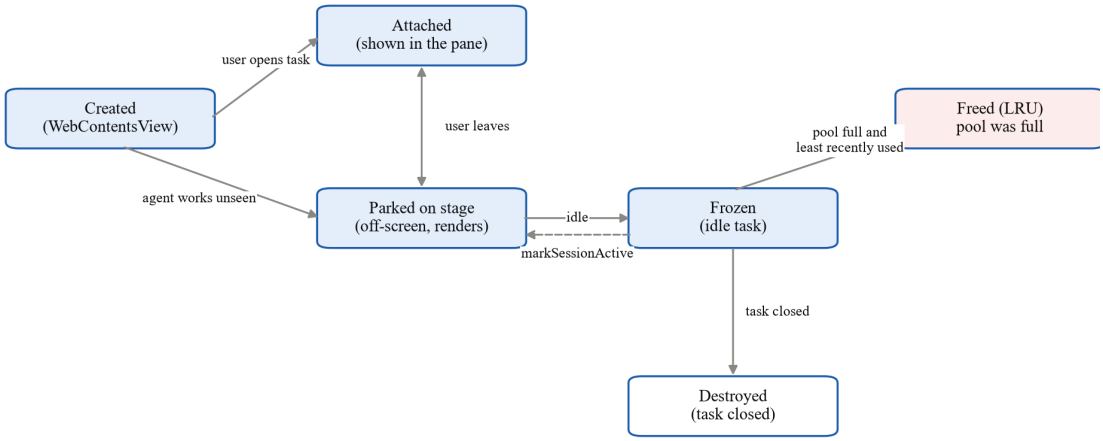


Fig. 5.3 Life cycle of a task's browser in the pool

5.1.4 Windows applications through UI Automation

Windows control is an MCP server (mcp-servers/windows) with 19 tools (Table 5.3). Its host is a long-running PowerShell process that compiles DexDesk.cs once and keeps it loaded. For each action the host tries, in order:

1. an operating-system or application interface, such as PowerShell and CIM, netsh, SystemParametersInfo or the media-session API;
2. a UI Automation pattern (Invoke, Value, Toggle, Selection, Expand or Scroll) on the target window, wherever that window is;
3. borrowed input.

Table 5.3 The 19 tools of the Windows control server

Group	Tools
Read (never asks)	windows_list, window_tree, window_find, window_capture, ui_wait, system_info
Act inside an app through UI Automation	ui_invoke, ui_set_text, ui_toggle, ui_select, ui_expand, ui_scroll
Open and arrange	app_launch, window_manage, open_settings, media
Change the system (allowlist, journaled)	system_change, undo
Borrowed input (last resort)	input_act

Every UI Automation client sets IUIAutomation2.AutoSetFocus to false, because with the default setting even reading a background window can pull it to the front. Windows that DEX opens start without activation and are parked off-screen with SetWindowPos. Captures use PrintWindow into a bitmap sized for the window's own DPI, so an application that is not DPI-aware is still captured whole.

DEX moves only the windows it opened, and it acts on the user's own windows in place through patterns. The Desk tab shows live frames of the window DEX is working in while someone is watching it, and the "Show me" and "Hand back" buttons move the window to the user and back. Chapter 6 shows a real task working in Notepad this way (Fig. 6.3).

5.1.5 Borrowed input that gives way to the user

Some steps need real input, for example a shortcut that an application accepts only from the keyboard, or a canvas with no accessibility support. For these the host borrows the mouse and keyboard (Fig. 5.4). It first checks that the target is not a password field, that the user has been idle for at least 1.5 seconds, and that no full-screen video, game or presentation is in front. If a check fails, the call returns a reason such as user_busy, user_fullscreen or secret_field, and the engine waits or finds another way (Appendix A.2).

When the checks pass, the host records the foreground window and the cursor position, brings the target into place and sends input with SendInput [23], setting the dwExtraInfo field of every event to 0x44455821 ("DEX!" in ASCII). Low-level mouse and keyboard hooks watch all input during the borrow (Appendix A.1). An event without that value came from the user, so at the first one the host stops, restores the window, the foreground and the cursor, and reports user_took_over. A thin glow along the edge of the screen shows while input is borrowed, and

when the step finishes, everything is restored and the window is parked again.

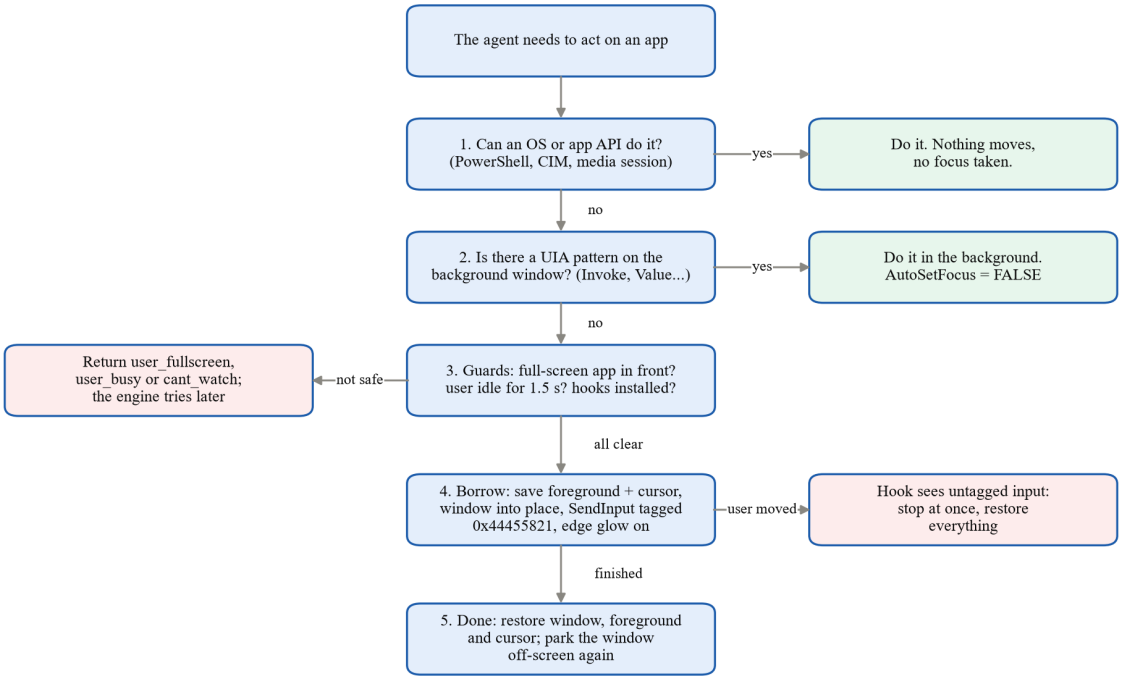


Fig. 5.4 How DEX picks a background interface, and when it borrows input

5.1.6 Windowless helpers and system changes

Blender runs per task as a background process with no window. A small host script drives Blender's Python API, renders the camera view in place of a viewport screenshot, saves the scene after changes and quits after 15 idle minutes. DEX writes Office documents directly as files and opens PDFs and spreadsheets in its own document tabs.

System changes go through a closed allowlist of 25 actions, and each change is recorded in an undo journal before it runs. Ten of the actions are reversible, for example flushing DNS, restarting Explorer, disabling a startup app or cleaning temporary files. Three are hard to undo: resetting an app, emptying the Recycle Bin and uninstalling an app. The other twelve need administrator rights, such as renewing the IP address, restarting a network adapter or service, or creating a restore point. These go through a helper that is set up once with a single elevation prompt, and the agent itself never runs elevated. In Settings > Desktop control (Fig. 5.5) the user sets per-app rules and can see the off-limits list and the recent changes, each with an Undo button.

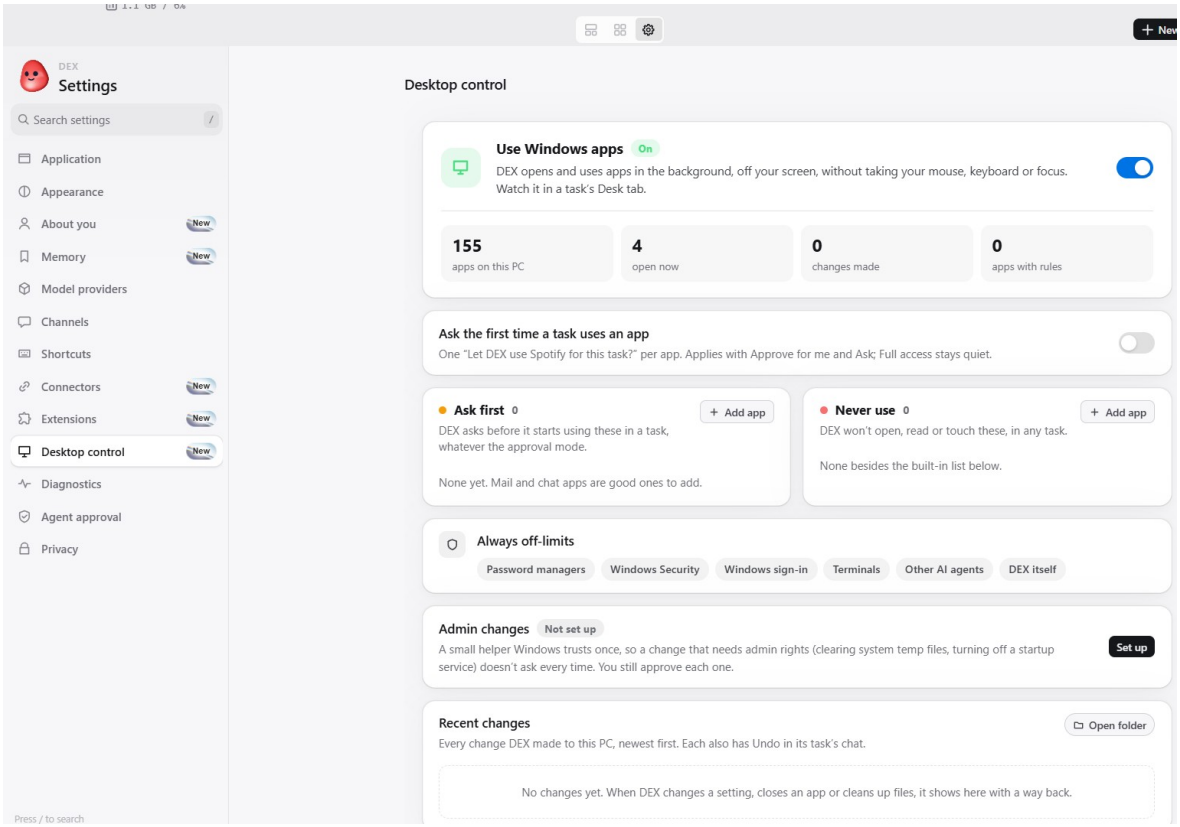


Fig. 5.5 Settings > Desktop control, with background app use, per-app rules, the off-limits list and recent changes

5.1.7 Chrome extensions for the agent

Some sites depend on an extension, for example a library proxy, a reference manager, a translation tool or an ad blocker. DEX's browser runs real Chrome Web Store extensions. It verifies their CRX3 packages itself, provides the chrome.tabs and chrome.windows APIs through a shim, and implements declarativeNetRequest in the main process. Extensions load into the shared web profile and never into DEX's own windows.

A dex-ext tool lets the agent list the installed extensions, press an extension's button, open its menu or options, and install a new one from a store link once the user confirms. When a button opens a popup or side panel, DEX opens it as a background tab of the task, so the page stays the extension's active tab and the popup shows the right site. Every prompt names the installed extensions. Password managers and cryptocurrency wallets are refused entirely, including their pages, frames, buttons and menu items.

5.1.8 Connectors through the Model Context Protocol

The Marketplace in Settings > Connectors (Fig. 5.6) lists built-in accounts and 51 hosted MCP servers. Google (Gmail, Calendar, Meet, Drive, Docs, Sheets, Contacts and Tasks) is served by DEX's own MCP server, while Microsoft 365, GitHub, Slack, Reddit, Hugging Face

and Discord sign in through their own OAuth flows.

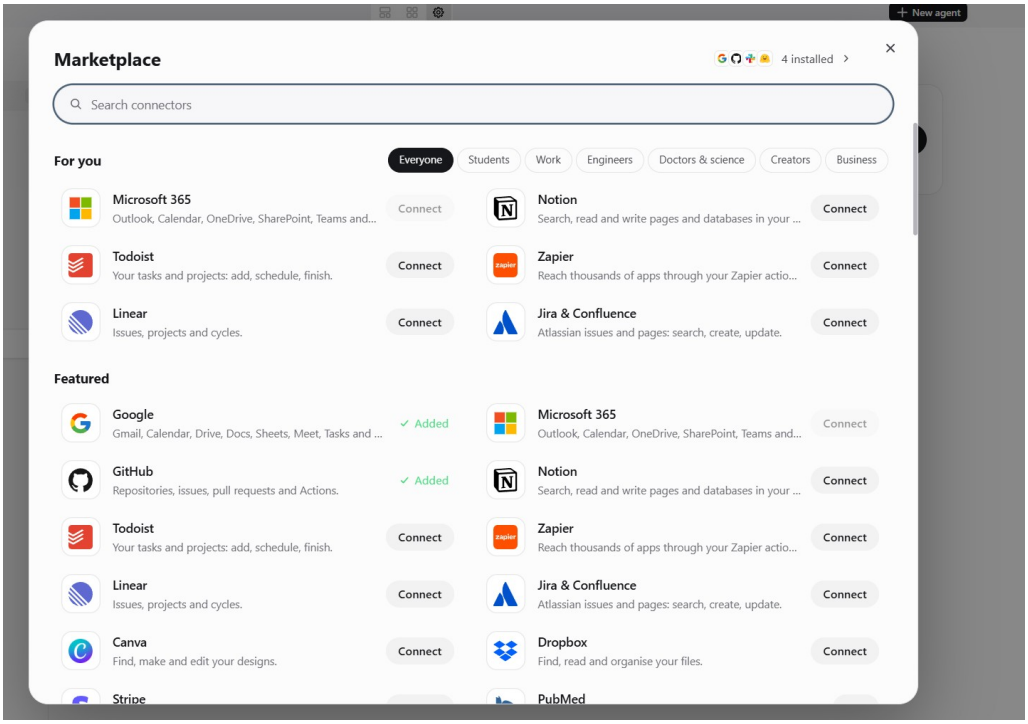


Fig. 5.6 The connector Marketplace

Of the hosted servers, 42 use OAuth and 9 need no sign-in. They follow the MCP authorisation specification (Fig. 5.7). DEX finds the authorisation server through protected-resource metadata [27] and server metadata [28], registers itself with a loopback redirect [26] and signs in with PKCE [25], naming the MCP server as the token's audience. Tokens are kept in the credential store and refreshed before each run, and a small bridge connects an engine's standard-input MCP transport to the server's streamable HTTP transport. A prompt that mentions flights, hotels or travel automatically gets the Kiwi.com flight server, the trivago hotel server or both, and neither needs a sign-in.

5.1.9 Generative-UI answers, triage and approvals

DEX answers with widgets where it can. With the dex-ui tool the agent can ask questions through pickers for places, dates and times, show results as cards with buttons, list facts or draw charts. The desktop renderer, the Android app and the chat adapters share one schema, so the same answer appears as buttons on Telegram and Discord and as numbered choices on WhatsApp, and a tap comes back as the user's next message.

When asked to reply to a batch of messages, DEX shows a triage card. Each message is a point placed by urgency on the horizontal axis and importance on the vertical axis, so the four corners stand for Reply now, Quick reply, Plan a reply and Later. Opening a point shows the message with a drafted reply and Yes, No and Specify buttons.

An approval card states what kind of step is waiting. The user can deny it, or approve

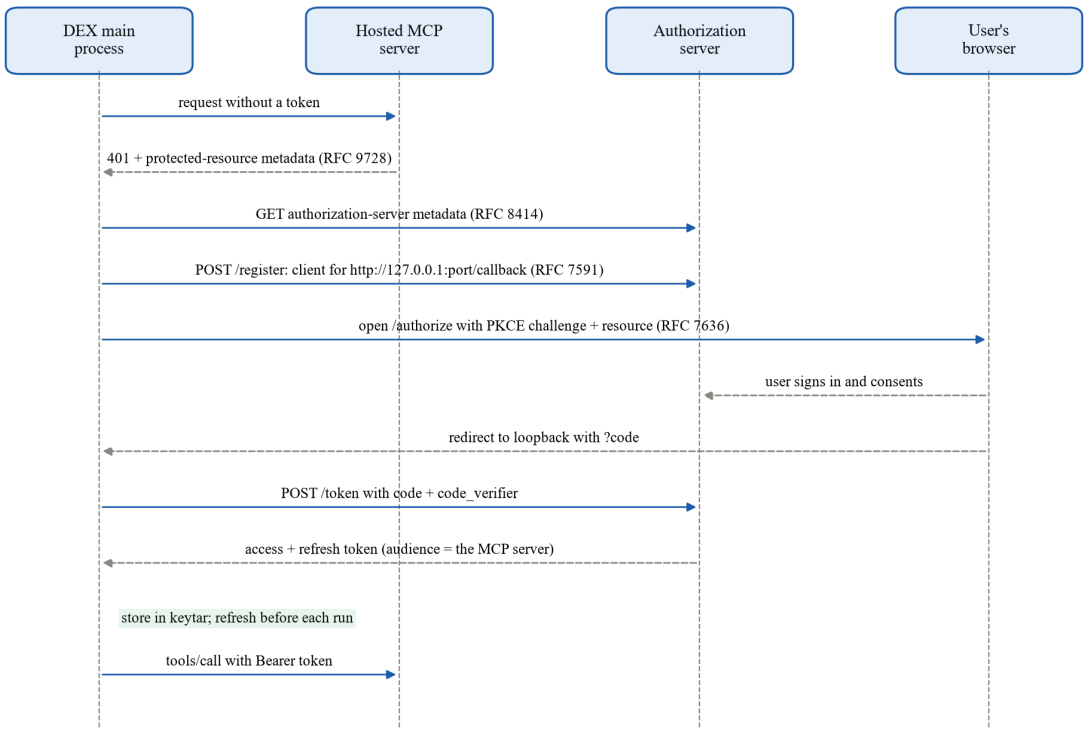


Fig. 5.7 Signing in to a hosted MCP server with discovery, dynamic registration and PKCE

it once, for the current turn or for the session. The decision is made outside the engine in four ways.

- A Windows or system call that needs approval blocks in the tool server while it asks the main process for a card, and the engine can neither skip the wait nor answer the card.
- Each action has a category, such as app control, system change, hard to undo or administrator rights. Approvals for the last two are never remembered, so each such action asks again.
- DEX's Windows tools refuse any window that belongs to DEX itself, so the agent cannot press its own approval card.
- Every desktop call that changes something first checks a per-task hold, so pressing Pause on the PC or the phone stops desktop actions at once.

There are three global modes (Appendix A.4). "Ask for approval" stops at every sensitive action. "Approve for me" lets DEX work inside apps without asking, but it asks before system changes, risky paths and risky commands. "Full access" asks only before Windows registry writes, which always need approval. Full access is the default, so a fresh installation shows approval cards only after the user picks a stricter mode. Two further guards from the design have not been built yet (Section 8.2): approvals signed with a hash of the exact operation, and a rule on the desktop cards that requires real pointer movement, like the one in the prototype.

5.1.10 Memory, site memory and routines

User memory is a set of notes grouped by topic and stored in the application data folder, outside the shipped code. An optional "about you" questionnaire fills it in at the start (Fig. 5.8), and history can be imported from ChatGPT or Claude. To keep a growing memory out of every prompt, a recall index matches the words of each task against the notes, saved sites and routines. The task receives the core notes, a few matches and a one-line outline of everything else, and the agent can fetch more with a dex-recall tool.

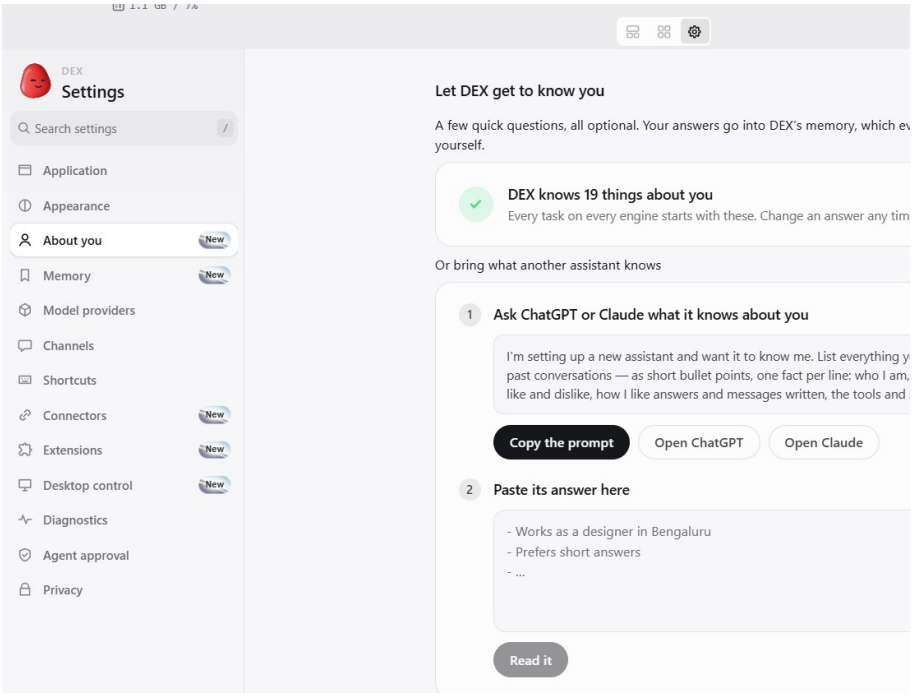


Fig. 5.8 Settings > About you, with optional questions that seed DEX's memory and an import from another assistant

Site memory keeps one Markdown note per website that describes its pages, navigation and controls. The agent reads the note before browsing the site, so it can open a mapped portal directly at the right page, and the /scrape command builds such a note from a site's public structure. Site aliases such as "university portal" are stored in a plain file. Site logins go into the credential store, and DEX types them straight into the page, so the agent never sees the password.

Routines are written in plain words, for example "every morning at 7:30, check my mail and calendar". A dex-workflow tool saves each one as a validated specification, and a scheduler checks for due routines every 30 seconds. A run missed while the PC was asleep still starts if it is less than three hours late. The /workflow command lists the routines as cards, and each card's View button opens a step-by-step graph. Two early failures shaped this part of the design: the engine first used its own cloud scheduler instead of DEX's, and Windows PowerShell added an invisible byte-order mark to piped JSON.

5.1.11 Security review: /bugbounty

/bugbounty <url> expands into a full instruction backed by a skill file. The agent must first confirm that the target belongs to the user, is in a published bug-bounty scope or is explicitly permitted. It then reviews the site without changing anything, looking at:

- security headers and TLS configuration;
- exposed conventional paths such as /.git or /.env;
- versions of client-side libraries visible on the page;
- form and cookie hygiene, meaning CSRF tokens, the Secure, HttpOnly and SameSite flags, and autocomplete on sensitive fields;
- information disclosure in comments, errors and source maps.

The instruction excludes exploitation, injection, authentication bypass, brute force and denial of service. Findings are saved as a security note beside the site's memory so that a later review can compare them. The boundary is written into the instruction and is not enforced by a tool, so it holds only as long as the engine follows the instruction.

5.1.12 Phone app and chat channels

DEX for Android is a native Jetpack Compose app in Material 3 Expressive with spring-based motion. It pairs with the PC through Firebase Authentication, using the same account on both, and Cloud Firestore carries devices, task transcripts and commands under the user's account. From the phone the user can start a task, watch it step by step, steer it, approve steps from a notification, run routines, answer widgets and triage cards, and view a 3D scene rendered on the PC. The app's screens are in Fig. 5.9 and Fig. 5.10.

Settings > Channels (Fig. 5.11) lists the phone, WhatsApp, Telegram and an advanced section. Telegram uses the user's own bot, created with @BotFather and long-polled from the PC without a server, and its pairing link accepts only the first person who opens it. WhatsApp links DEX as a device, so the user can message themselves "@DEX ...". Discord is a connector signed in with OAuth that posts to channels the user picks, and a bot of the user's own can be added under Advanced.

Widgets become inline buttons on Telegram, components on Discord and numbered answers on WhatsApp. The Telegram and Discord adapters have unit tests against fake APIs, but they have not been tested with real bots.

5.1.13 3D design in the background

For a Blender task, the agent works through the Blender MCP server in a Blender instance with no window, renders the scene and shows the result in the task's 3D viewer. For

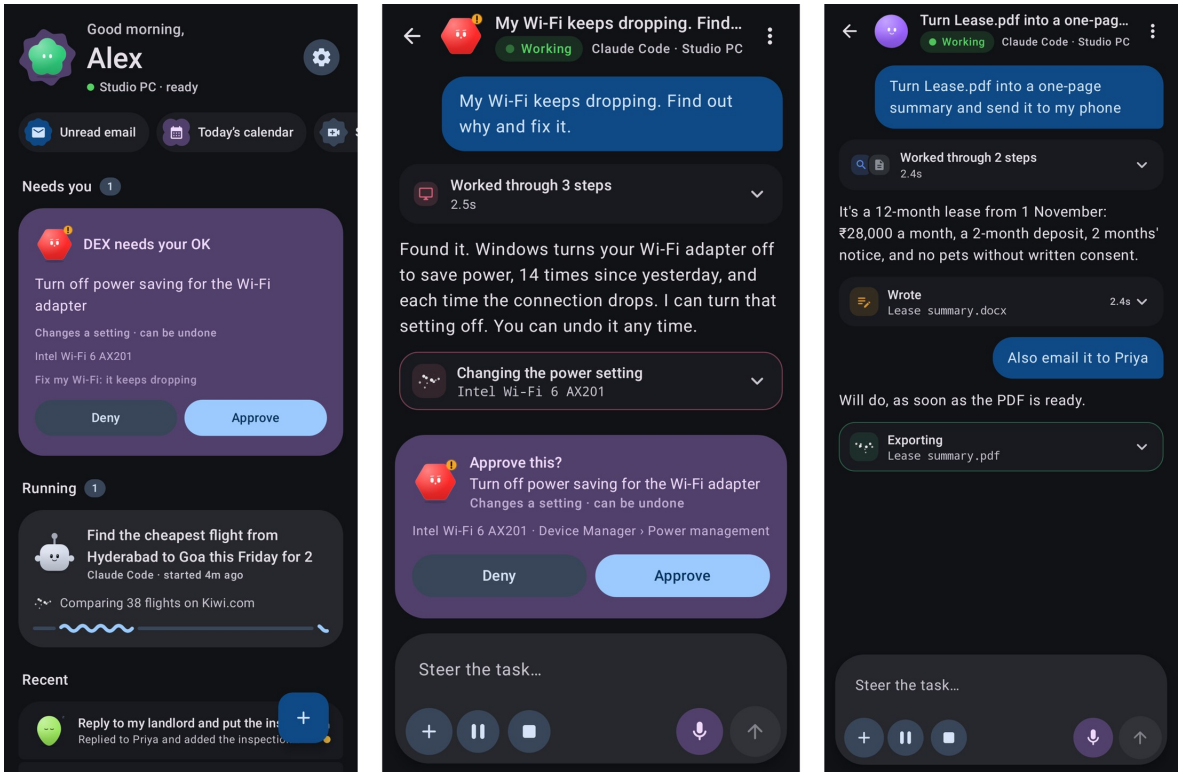


Fig. 5.9 DEX for Android, rendered from the app's code with sample data: home, answers as cards, and an approval

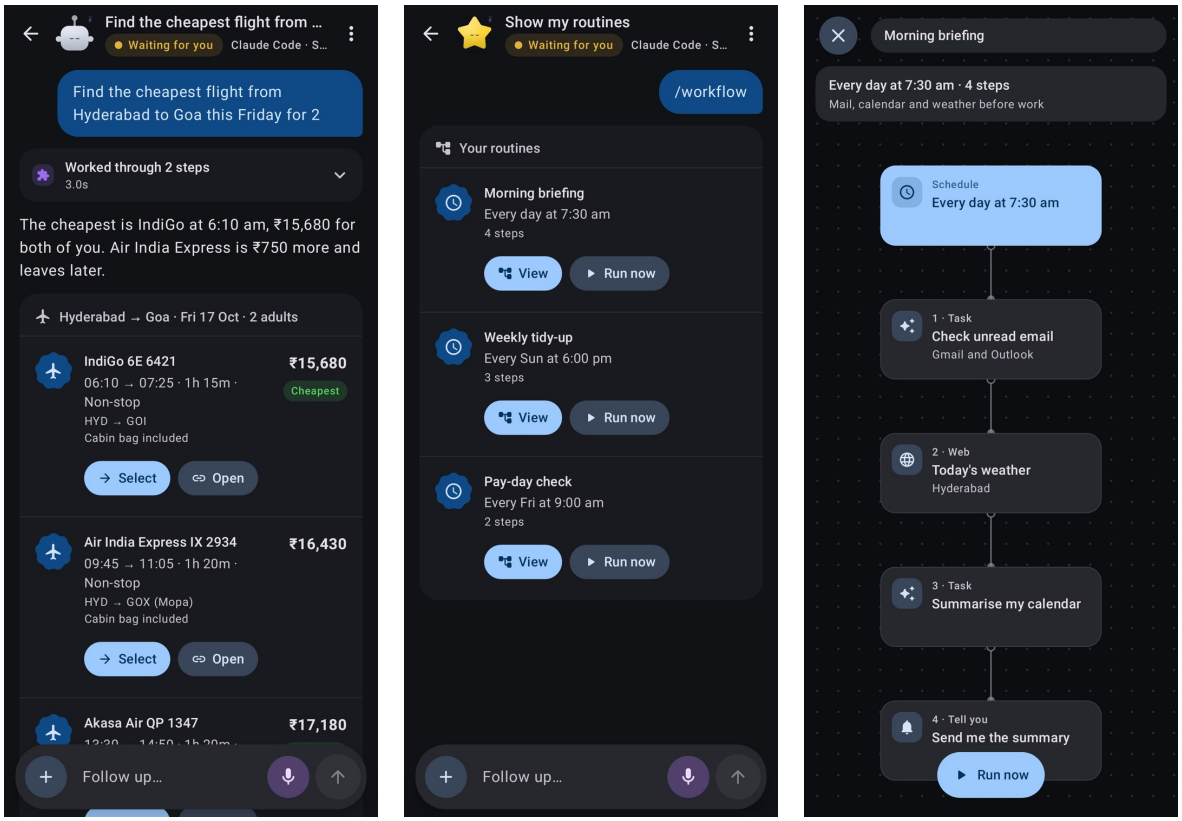


Fig. 5.10 DEX for Android, rendered with sample data: a live task, routines, and a routine shown as a graph

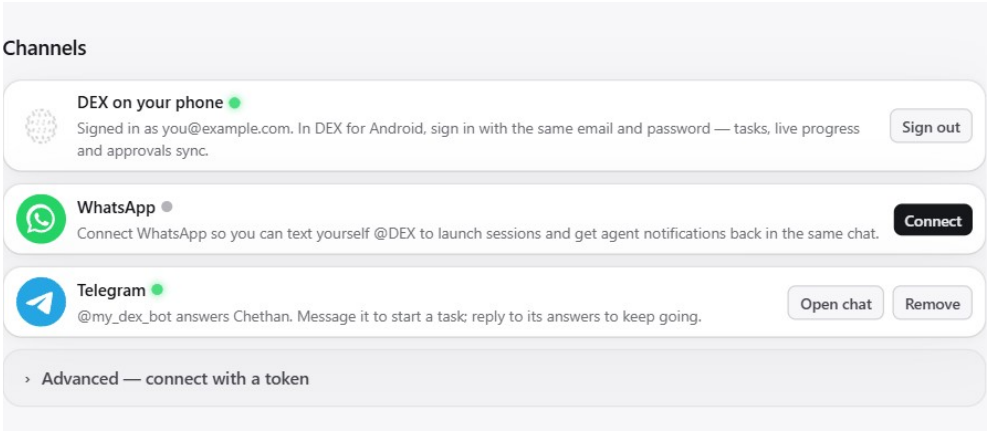


Fig. 5.11 Settings > Channels, with the phone, WhatsApp, Telegram and advanced options

objects it cannot model from primitives, dex-3d turns text into an image with FLUX.1-schnell [31] and the image into a 3D model with Hunyuan3D [32], using the user's own Hugging Face account. On the phone, the PC produces a render, a GLB model and the camera view for previewing a scene.

5.2 Testing

Each subsystem has unit tests written for Vitest. They cover, among other things, the browser pool's eviction and the CDP broker's refusals, the extension off-limits matching and the approval policy, the parsing of each engine's event stream, the conversion of widgets for each chat app, and the workflow scheduler. We ran the full suite on 21 October 2026. Of 1,342 tests in 156 files, 1,333 passed, 8 were skipped and 1 failed. The failing test checks that the Desk tab sends live frames only while someone is watching. It passed in two separate runs on its own, which points to a timing-sensitive test under the load of the full suite, though it still needs fixing.

We tested Windows control live against a PowerShell fixture window with common controls and learned two things on the way. A mouse move of zero distance never reaches a low-level hook, so a simulated user has to move at least one pixel, and screenshot scripts have to be DPI-aware. When no device was attached, Android screens were rendered to images with Robolectric. Settings > Diagnostics (Fig. 5.12) checks on any PC that the tools DEX needs are present, that the engines are signed in and that the network is reachable.

For end-to-end evaluation we built DEX-Bench (benchmarks/dex-bench in the repository). It uses OSWorld's method [6] of scoring the end state with a script, on a real Windows PC instead of a virtual machine, and has 24 tasks in six categories (Table 5.4). The web tasks run against a local, offline test site with a shop, a contact form, a data table, a student portal, a paginated library and a settings page behind a cookie banner. Each task has a setup step and a checker that reads the end state, either from files on disk (CSV, .docx, .xlsx, .zip and .png) or

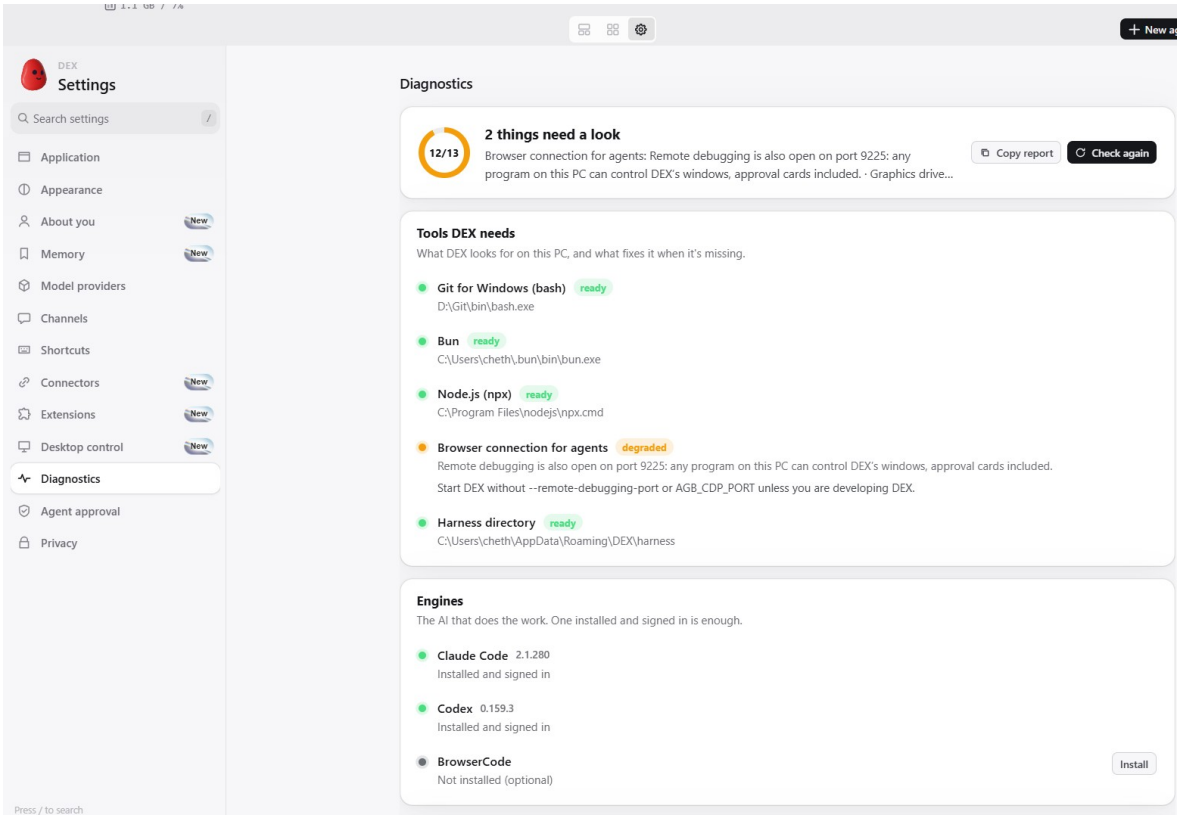


Fig. 5.12 Settings > Diagnostics, with tools, engines, network and warnings

from the test site's records of orders, form submissions, sign-ins and saved settings. Answers to questions are checked against the true value, and any route counts, whether it is clicking, a shell command or a script. A self-test runs each task's reference solution, which must pass, and an empty attempt, which must fail. On 21 October 2026 it reported all 24 checkers correct.

Table 5.4 The 24 DEX-Bench tasks

Category	Tasks	What the checker reads
Web (6)	web-cart, web-form, web-table, web-login, web-pages, web-settings	The test site's records (order, form, sign-in, settings), a CSV file, or the answer
Files & OS (5)	os-organize, os-rename, os-find, os-zip, os-sysinfo	Folders and file names, a copied file, the zip's contents, or the answer checked against this PC
Documents (4)	doc-create, xlsx-total, csv-clean, doc-edit	Heading style and bullets in .docx, cells and formulas in .xlsx, CSV rows
Windows apps (4)	app-notepad, app-calc, app-display, app-explorer	The saved note, and for the other three only the answer
Multi-app (3)	multi-web-xlsx, multi-reply, multi-chart	A spreadsheet of the shop's products, a reply file, a chart image
Infeasible (2)	infeasible-file, infeasible-web	That the answer declines and nothing was made up or changed

The runner (run.mjs) gives the same prompts to four configurations (Table 5.5). For DEX, it drives the hub through DEX's own interface over a debugging port opened only for the

benchmark, creates one task per prompt, and switches the approval mode to Full access for the run before restoring it. For each engine alone, it starts the command-line tool in the task's folder with the same model, with permission prompts bypassed and without DEX's prompt, tools or MCP servers. The runner waits out model usage limits instead of counting them as failures, and it gives each task up to 900 seconds by default.

Table 5.5 The four configurations run on DEX-Bench

Configuration	Engine and model	Run (UTC)
DEX, Claude Code engine	Claude Code CLI, claude-opus-5-5	20 Oct 2026, 10:48
Claude Code CLI alone	Claude Code CLI, claude-opus-5-5	21 Oct 2026, 06:21
DEX, Codex engine (two runs)	Codex CLI 0.159.3, gpt-6-luna, reasoning effort high	20 Oct 2026, 17:51 and 18:12
Codex CLI alone	Codex CLI 0.159.3, gpt-6-luna, reasoning effort high	21 Oct 2026, 05:53

DEX records the model of every task in its task database. We checked the Claude Code runs alone in Claude Code's own transcripts and the Codex runs in Codex's session logs. All runs were on the development PC described in Section 3.3.

CHAPTER 6

PROJECT DEMONSTRATION

The tasks in this chapter were run on the development PC and the phone between 7 and 21 October 2026. Each figure is a screenshot taken during the task, and the tool-call counts come from DEX's task database.

A task starts from the hub (Fig. 6.1) or from the phone. The hub shows how many tasks are running and idle and lists recent sessions, and the prompt box at the top starts a new task with the engine picked beside it.

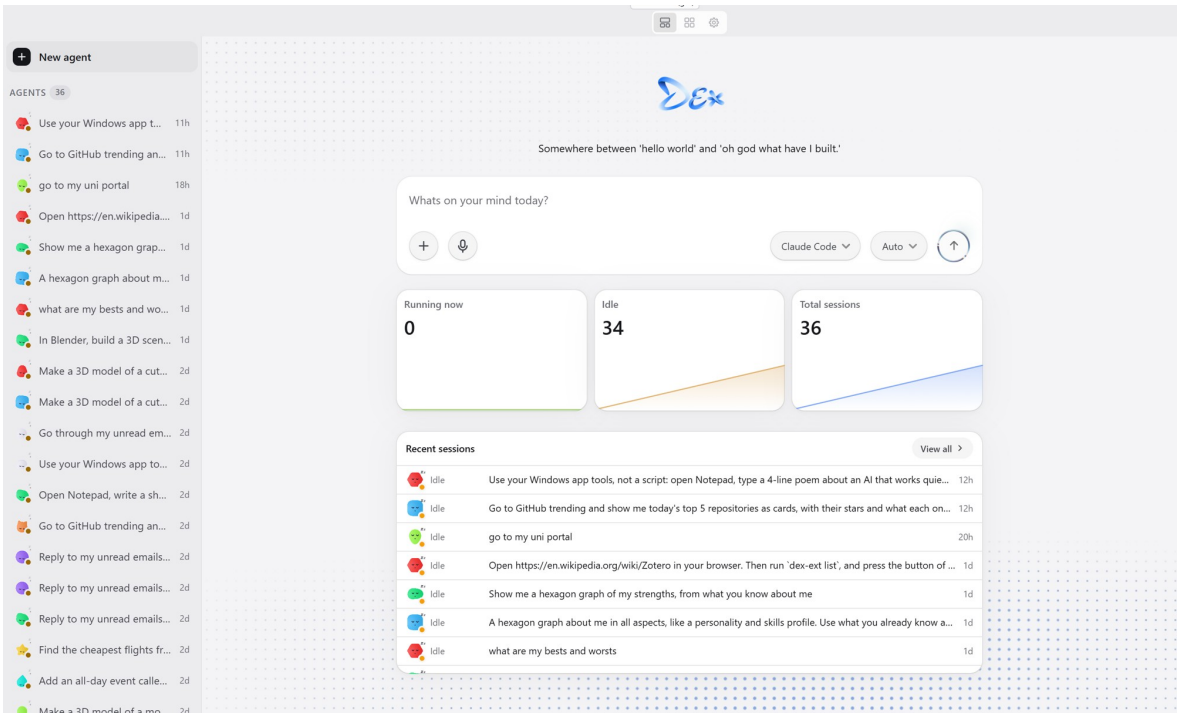


Fig. 6.1 The hub, with running tasks, tasks waiting for the user and recent tasks

Asked to go to GitHub Trending and show today's top five repositories as cards, DEX with Claude Opus 5.5 opened the page in the task's own tab (Fig. 1.1) while the tab showed a notice that the user could use the page at the same time. It answered with five repository cards that list their stars and descriptions (Fig. 6.2), in five tool calls.

Asked to write a four-line poem in Notepad with its Windows app tools instead of a script, DEX with Claude Opus 5.5 opened Notepad in the background and worked through its Save As dialog (Fig. 6.3). The task made 27 tool calls, among them app_launch, window_find, window_tree, ui_set_text and ui_invoke, and the Desk tab showed the window DEX was working in.

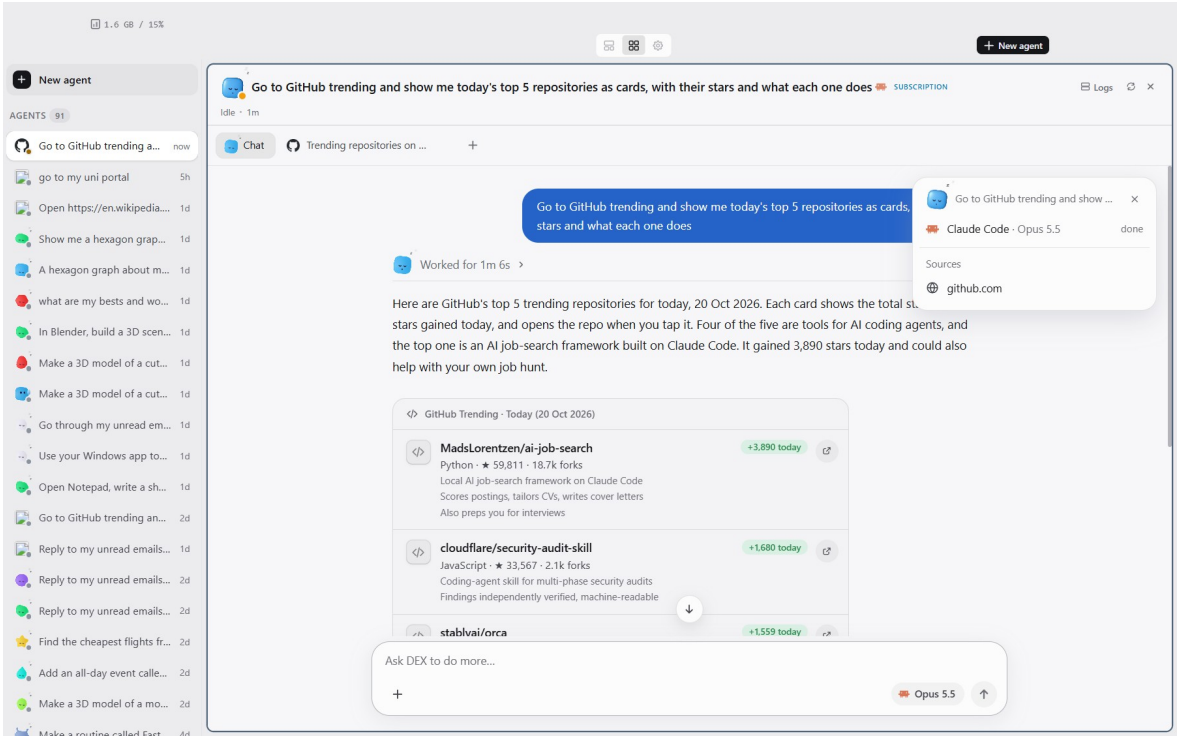


Fig. 6.2 The answer to the GitHub task, today's top repositories as cards with their stars and descriptions

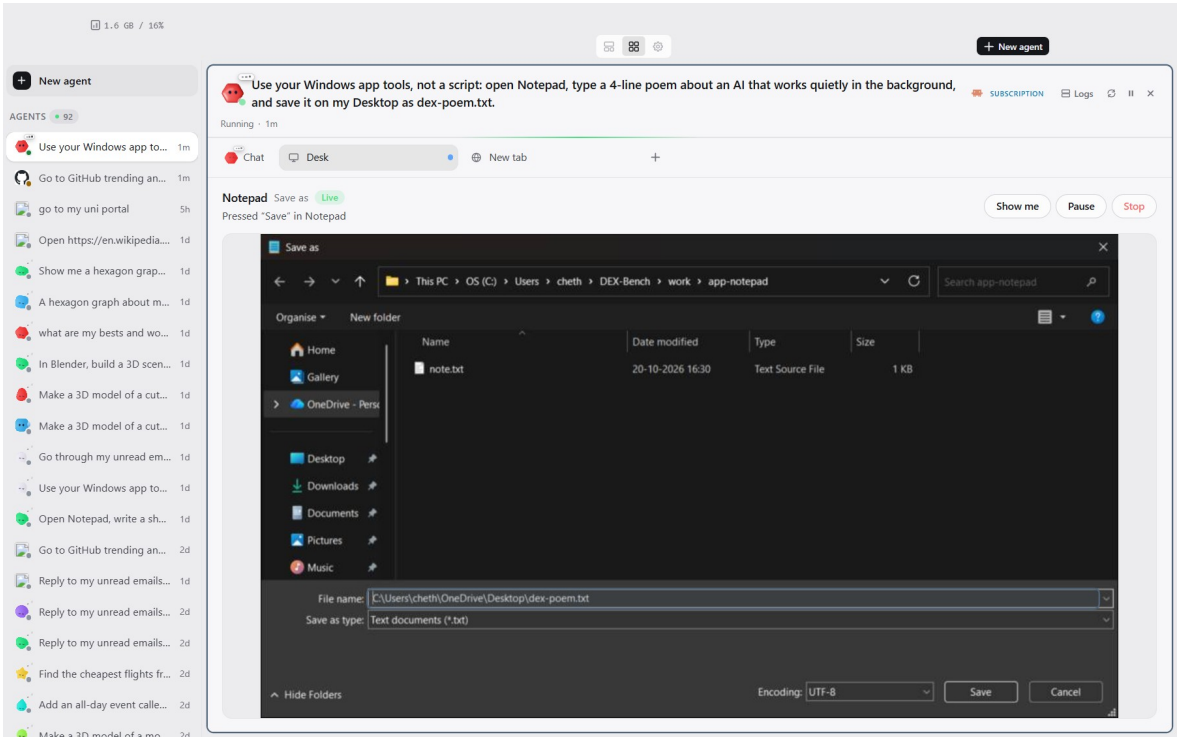


Fig. 6.3 A real task operating Notepad's Save As dialog through the Windows tools, seen in the Desk tab

Asked for the cheapest flights from Hyderabad to Goa on Friday 23 October 2026 for two adults, DEX with Claude Sonnet 5 used the Kiwi.com connector. It made five tool calls, three of them Kiwi.com searches, and answered in about 98 seconds with cards that have Select and Open buttons (Fig. 6.4). A separate task added an all-day event called “Review final” on 30 October 2026 to Google Calendar in three tool calls (Fig. 6.5).

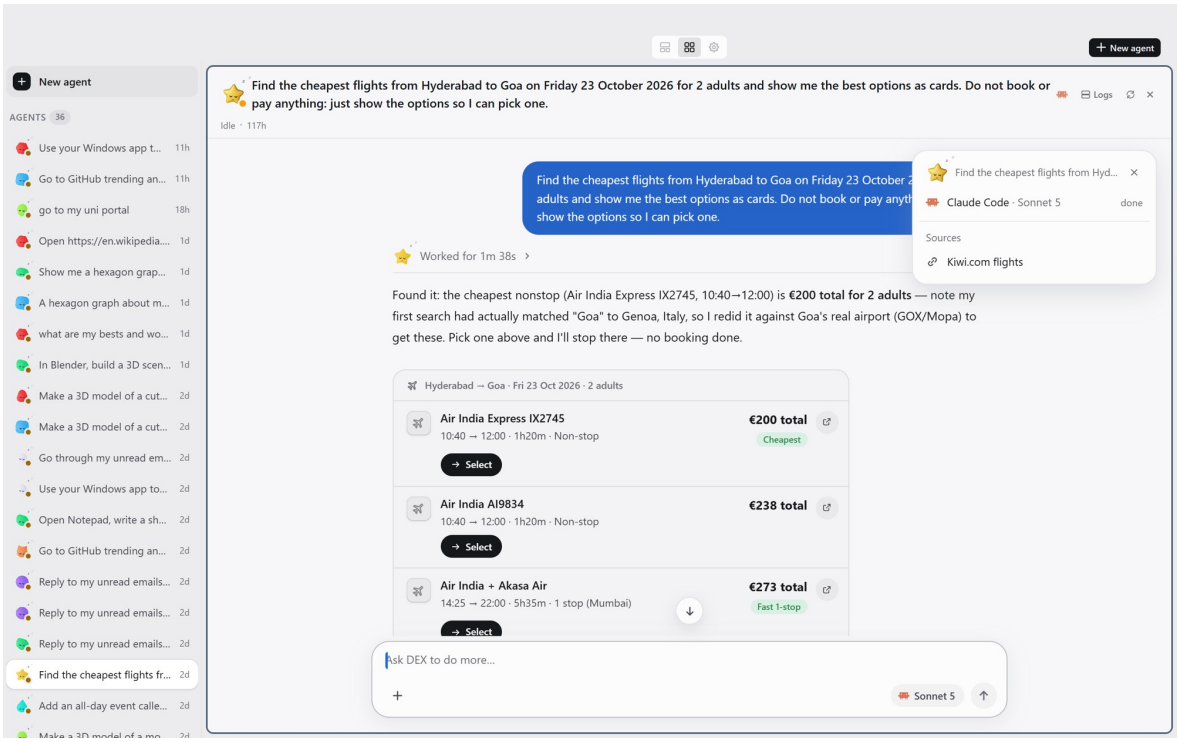


Fig. 6.4 A real flight search answered through the Kiwi.com connector, as cards with Select and Open buttons

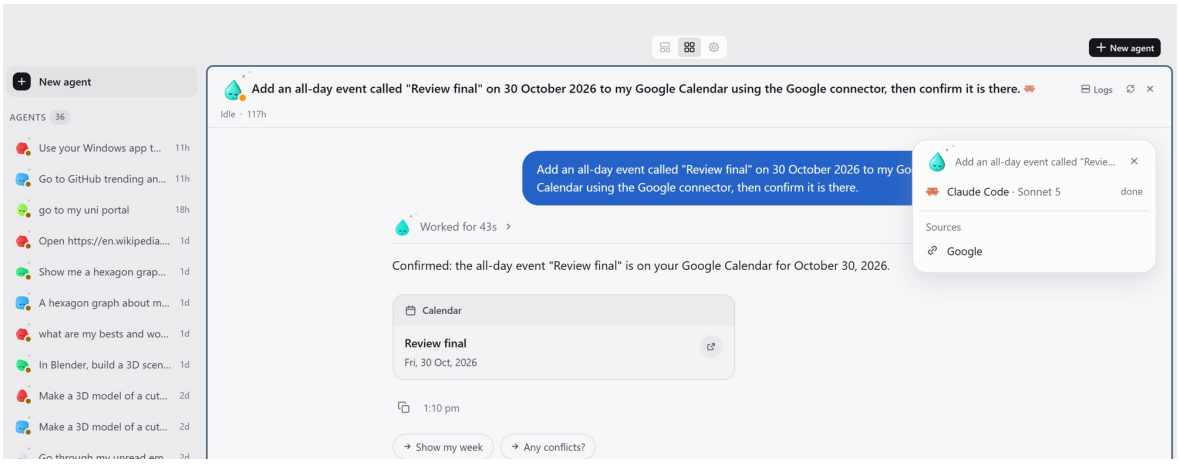


Fig. 6.5 A real task adding an all-day event through the Google Calendar connector

Asked to go through the unread mail of the last seven days and help reply, DEX with Claude Haiku 4.5 read the mail through the Gmail connector and placed each message on a triage card, in six tool calls (Fig. 6.6). Asked for “a hexagon graph of my strengths, from what

you know about me", DEX with Claude Opus 5.5 looked up the user's memory twice with dex-recall and drew a chart widget, in four tool calls (Fig. 6.7).

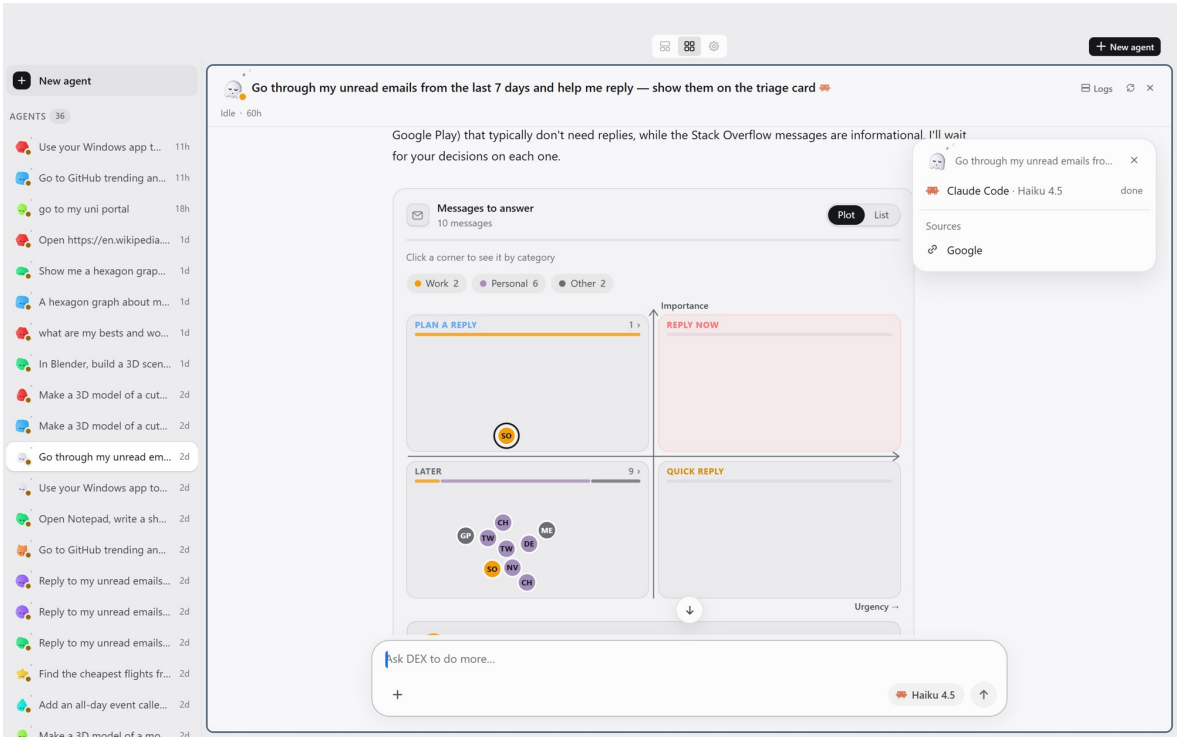


Fig. 6.6 A real triage card for unread mail, with urgency across, importance up and a drafted reply for each message

When a step needs approval, the card appears at the top of the task (Fig. 6.8). In this task, opening File Explorer in the background waited for the user's answer, and Section 7.7 describes what happened after the user denied it.

A task on the Wikipedia page for Zotero listed the installed Chrome extensions with dex-ext and pressed an extension's button, in nine tool calls (Fig. 6.9).

The Japanese shrine scene in Fig. 6.10 was built in a windowless Blender in one task with Claude Opus 5.5, over 120 tool calls.

From the phone, a search for the cheapest flights to Canada ran on the PC and returned its answer as cards on the phone (Fig. 6.11). It ran on 7 October, before the travel connectors existed, and drove a flight website through 20 browser calls out of 23 tool calls, taking about nine minutes. The Hyderabad to Goa search above took five tool calls through a connector. The two tasks used different routes and models, so they illustrate the difference and do not measure it.

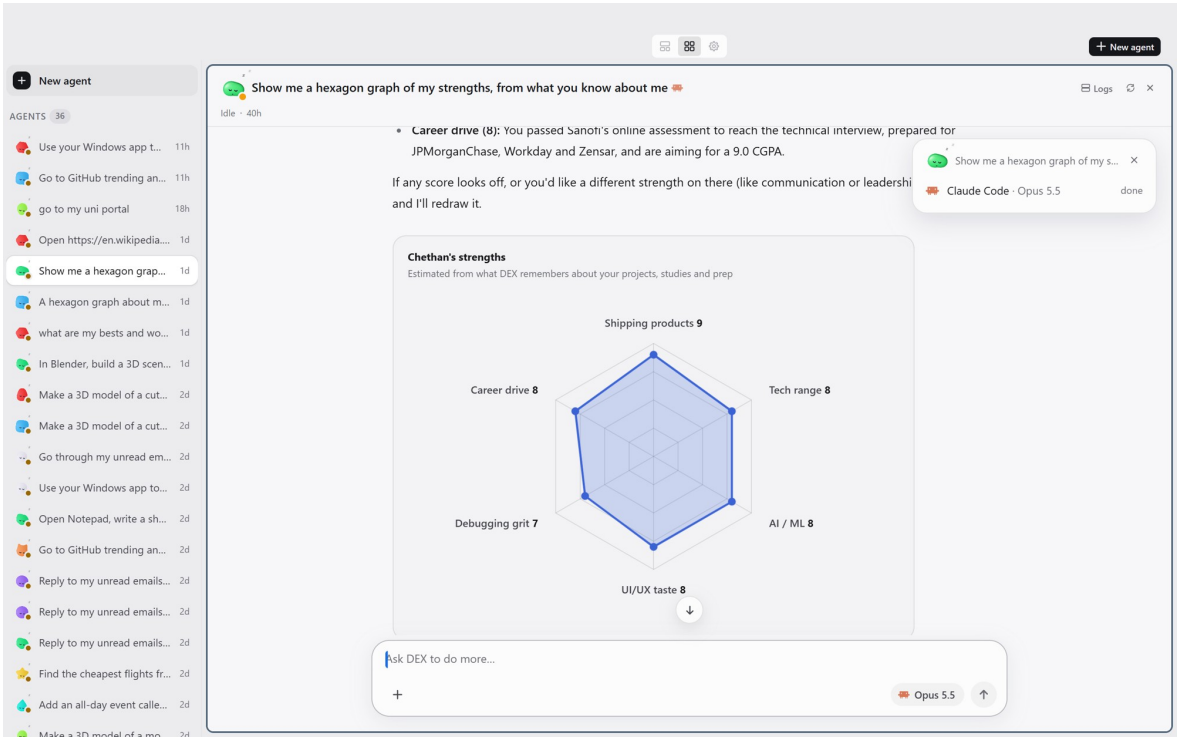


Fig. 6.7 A chart widget built from memory for the prompt "Show me a hexagon graph of my strengths, from what you know about me"

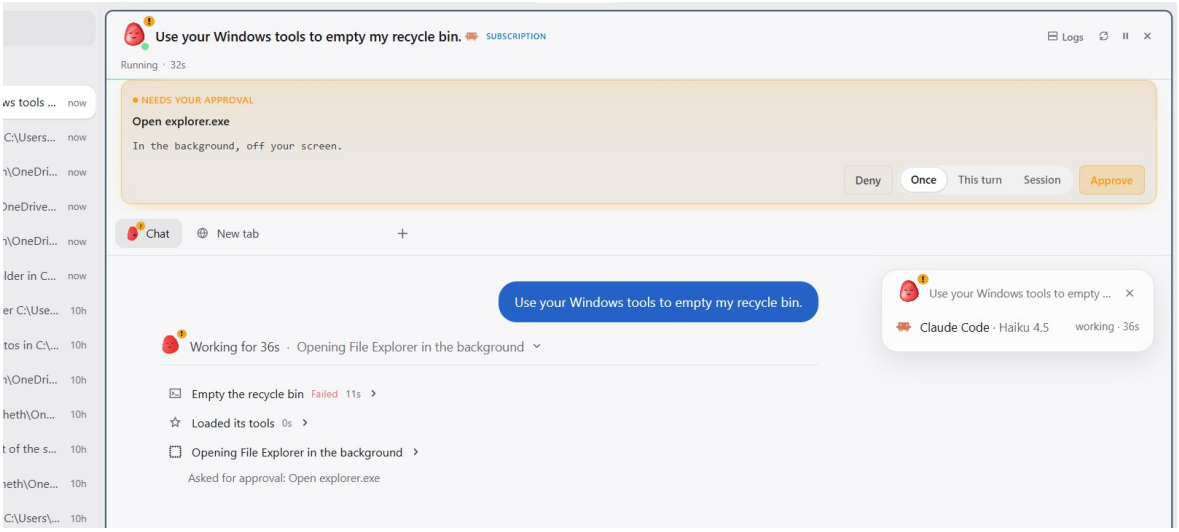


Fig. 6.8 An approval card for a background step, with Deny, Once, This turn, Session and Approve

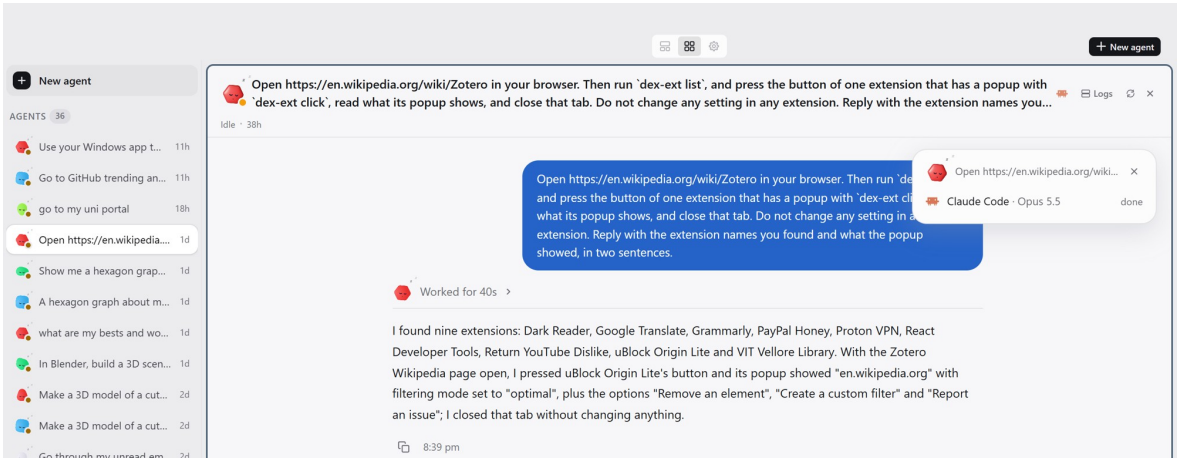


Fig. 6.9 A real task listing and using the installed Chrome extensions on a Wikipedia page



Fig. 6.10 A Japanese shrine scene modelled and rendered by DEX in a windowless Blender

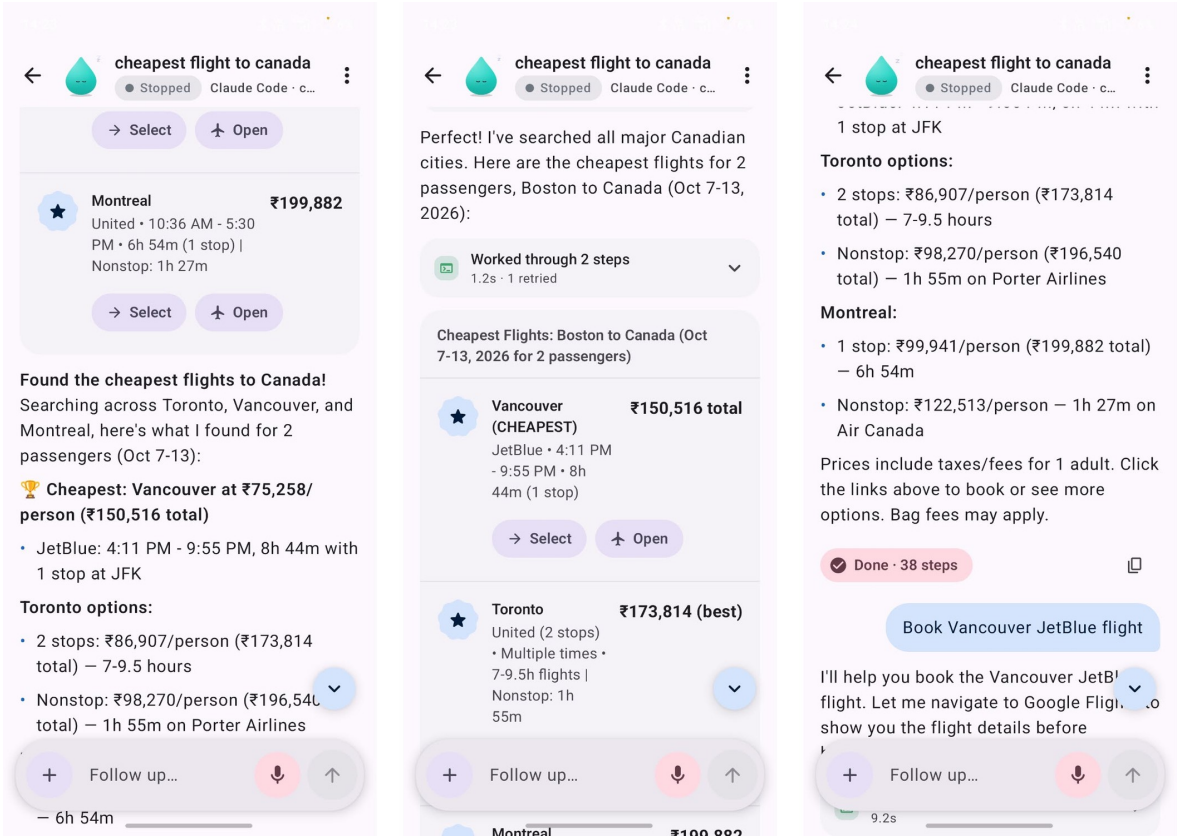


Fig. 6.11 Screenshots from a real phone: a search for the cheapest flights to Canada, started and answered on the phone (23 tool calls)

CHAPTER 7

RESULT AND DISCUSSION

7.1 DEX-Bench results

Table 7.1 and Fig. 7.1 give the results of the four configurations. With Claude Opus 5.5, DEX passed all 24 tasks with a median of 39 seconds and 4 tool calls per task, and the suite took 21.5 minutes of task time. With gpt-6-luna, DEX passed 20 of 24 tasks in each of two full runs, though a different set of tasks failed in each run.

Table 7.1 DEX-Bench results for each configuration

Configuration	Passed	Failed	Median time per task	Total task time	Cost reported
DEX, Claude Code engine	24/24	0	39 s	21.5 min	$6.24
Claude Code CLI alone	24/24	0	25.5 s	22.9 min	$6.15
DEX, Codex engine, run 1	20/24	4	46 s	21.2 min	$2.92 (estimated)
DEX, Codex engine, run 2	20/24	4	46 s	20.4 min	$2.68 (estimated)
Codex CLI alone	19/24	5	32.5 s	16.4 min	not recorded

Costs for the Claude Code runs are the API-equivalent cost that Claude Code reports, although the runs themselves used a subscription. DEX estimates the cost of the Codex engine from token counts. Steps are not compared across configurations, because each runner counts them differently: DEX counts tool calls, Claude Code reports turns and the Codex runner counts completed items.

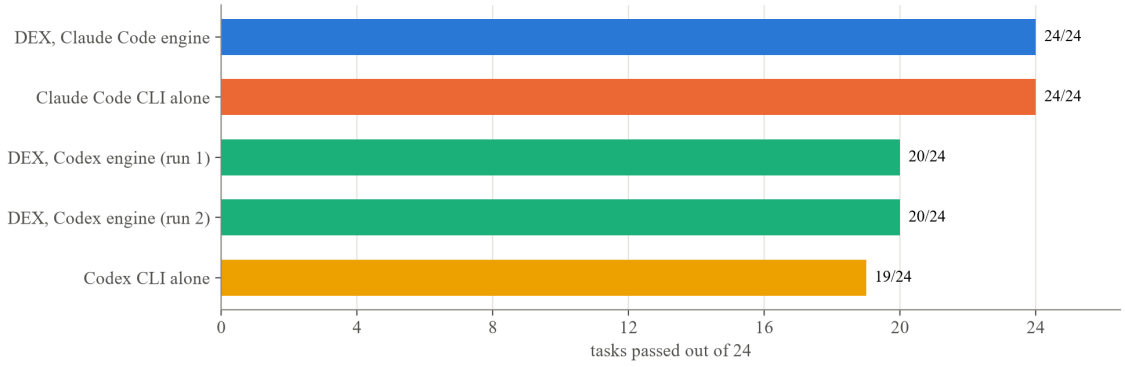


Fig. 7.1 Tasks passed by each configuration (Claude Opus 5.5 for the first two, gpt-6-luna for the rest)

Table 7.2 breaks the results down by category. Neither Claude configuration failed a task: DEX with the Claude Code engine and the Claude Code CLI alone both finished with 0 failures. The Codex CLI alone failed 5 tasks, and DEX with the Codex engine failed 4 in each

of its two runs, so the same model and CLI lost one task fewer when it ran inside DEX, in both runs. Most failures were in the Web, Documents and Multi-app categories. Files & OS and Windows apps each failed once, and the two infeasible tasks never failed. Fig. 7.2 shows every task in every run with its time.

Table 7.2 Tasks passed and failed by category

Category	Tasks	DEX, Claude Code	Claude Code alone	DEX, Codex (run 1 / run 2)	Codex alone
Web	6	6	6	4 / 4	4
Files & OS	5	5	5	4 / 5	5
Documents	4	4	4	3 / 3	3
Windows apps	4	4	4	4 / 4	3
Multi-app	3	3	3	3 / 2	2
Infeasible	2	2	2	2 / 2	2
Total passed	24	24	24	20 / 20	19
Total failed		0	0	4 / 4	5

	DEX, Claude Code engine	Claude Code CLI alone	DEX, Codex engine (run 1)	DEX, Codex engine (run 2)	Codex CLI alone
web-cart	98 s	38 s	70 s	55 s	**FAIL 91 s**
web-form	40 s	16 s	34 s	58 s	46 s
web-table	31 s	17 s	43 s	**FAIL 40 s**	29 s
web-login	40 s	14 s	**FAIL 31 s**	**FAIL 53 s**	49 s
web-pages	37 s	25 s	**FAIL 40 s**	40 s	**FAIL 13 s**
web-settings	55 s	29 s	49 s	73 s	50 s
os-organize	31 s	20 s	64 s	52 s	28 s
os-rename	31 s	23 s	34 s	31 s	24 s
os-find	49 s	13 s	31 s	31 s	15 s
os-zip	28 s	19 s	**FAIL 37 s**	34 s	43 s
os-sysinfo	16 s	10 s	28 s	21 s	13 s
doc-create	22 s	28 s	94 s	118 s	104 s
xlsx-total	55 s	61 s	**FAIL 100 s**	**FAIL 85 s**	**FAIL 145 s**
csv-clean	22 s	18 s	27 s	25 s	31 s
doc-edit	37 s	26 s	64 s	67 s	56 s
app-notepad	98 s	669 s	106 s	49 s	25 s
app-calc	258 s	43 s	25 s	21 s	**FAIL 12 s**
app-display	48 s	26 s	18 s	21 s	16 s
app-explorer	97 s	16 s	97 s	27 s	17 s
multi-web-xlsx	59 s	44 s	94 s	**FAIL 136 s**	**FAIL 44 s**
multi-reply	22 s	28 s	18 s	25 s	14 s
multi-chart	38 s	58 s	58 s	55 s	46 s
infeasible-file	26 s	110 s	52 s	43 s	34 s
infeasible-web	50 s	20 s	55 s	61 s	41 s

Fig. 7.2 Every DEX-Bench task in every run, with its time in seconds and failures marked

Table 7.3 gives the reason for each failure from the checker's record. Two failures recur.

xlsx-total failed in every gpt-6-luna run, with and without DEX, because the checker found neither a "Total" header in D1 nor the row totals. web-login failed in all three DEX Codex-engine runs, a partial re-run included: the answer gave the right exam date, but the test site recorded no sign-in. In the second run the agent reported that the portal "was already signed in". All of DEX's task tabs share one persistent web profile and the test site's sign-in cookie has a fixed value, so a cookie left by an earlier task or run would let the agent reach the dashboard without signing in. That is the likely cause, but we did not re-run with a cleared profile, so the task counts as failed.

The Codex CLI alone lost four of its five tasks without really attempting them. It stopped to ask for approval twice (web-pages and app-calc) and ended after a single step twice (web-cart and multi-web-xlsx). Neither of these failure types appeared in DEX's Codex runs, where every failure came from a wrong output or from the sign-in problem above.

Table 7.3 Why each failed task failed, from the checker's record

Configuration	Task	What the checker found
DEX, Codex, run 1	web-login	Correct date in the answer, but no sign-in recorded by the site
DEX, Codex, run 1	web-pages	Answered 6, while the catalogue lists 7
DEX, Codex, run 1	os-zip	fig1.png zipped at the top level instead of under figures/
DEX, Codex, run 1	xlsx-total	No Total header, row totals or grand total
DEX, Codex, run 2	web-table	Populations split across columns (digit-group commas written as CSV separators)
DEX, Codex, run 2	web-login	The agent reported the portal was already signed in, and no sign-in was recorded
DEX, Codex, run 2	xlsx-total	No Total header, row totals or grand total
DEX, Codex, run 2	multi-web-xlsx	Spreadsheet with no header and no product rows
Codex CLI alone	web-cart	No order placed (ended after one step)
Codex CLI alone	web-pages	Stopped to ask for approval before opening the page
Codex CLI alone	xlsx-total	No Total header, row totals or grand total
Codex CLI alone	app-calc	Stopped to wait for approval before using Calculator
Codex CLI alone	multi-web-xlsx	No products.xlsx (ended after one step)

Two partial re-runs of the Codex engine, of 11 and 7 tasks, passed 10 and 6. Their failures were again web-login and xlsx-total, and they are not counted in the totals above.

7.1.1 Checker corrections and re-scoring

We corrected two checkers after some of the runs and re-scored the saved answers with rescore.mjs, which re-checks the tasks whose checker reads only the answer.

- web-pages. The original checker took the number before the word "books", so it read a correct reply such as "7. All 4 pages list 38 books, and 7 of them are by Ursula K. Le Guin" as 38. The prompt asks for just the number, so the checker now takes the first number in the reply.

- infeasible-file. The pattern that recognises a refusal matched only straight apostrophes (couldn't), and the gpt-6-luna replies used typographic ones (couldn't). It now accepts both.

Each correction changed exactly one verdict per configuration, always from fail to pass, and we read each of those replies and confirmed that it was right. Under the original checkers the scores would have been 23/24 for DEX with Claude Code, 23/24 for Claude Code alone, 19/24 for each Codex-engine run and 18/24 for Codex alone, in the same order as now.

DEX with Claude Code passed app-calc on its answer without operating Calculator. Calculator came to the foreground when it was relaunched, the take-over guard treated it as the user's window, and DEX worked out the product another way (Section 7.8). The checkers for app-calc, app-display and app-explorer read only the answer, so they cannot tell whether the app was used.

7.2 DEX compared with its engines run alone

The baselines show what DEX adds to the engine it wraps. With Claude Opus 5.5, both configurations passed every task, so there was no room for DEX to raise the score. With gpt-6-luna, DEX failed one task fewer than the Codex CLI alone in both runs (4 against 5), and four of the CLI's lost tasks were ones it never really attempted. On this benchmark, then, DEX was at least as reliable as either engine on its own and more reliable than Codex alone. The margin is one task out of 24, about four percentage points, so more runs are needed before the difference can be called firm.

DEX was slower than Claude Code alone. The median task took 39 s in DEX and 25.5 s without it, and DEX was slower on 18 of the 24 tasks (Fig. 7.3). Total task time came out close (21.5 against 22.9 minutes) only because Claude Code alone spent 669 s on app-notepad. We have not profiled where DEX's extra time goes. Likely causes are the larger wrapped prompt, starting the task through the hub, the checks made by the task's browser and the Windows tools, and, in app-calc, the take-over guard described in Section 7.8.

The two configurations also worked differently, which the scores do not show. With only a shell, the agents drove Windows from scripts. In app-notepad, Claude Code alone reported that its first attempt sent simulated key presses to the user's Chrome window, which was in front with a video open. Ctrl+N opened a new Chrome window, the text went into its address bar and Ctrl+S opened Chrome's Save As dialog, after which the agent closed them and finished the task another way. DEX's background design exists to prevent this kind of interference. DEX did the same task through its Windows tools (app_launch, ui_set_text and window_find) and did its web tasks in its own browser tabs. Because the benchmark scores only the end state, this difference does not show in the results, and it comes from a single run.

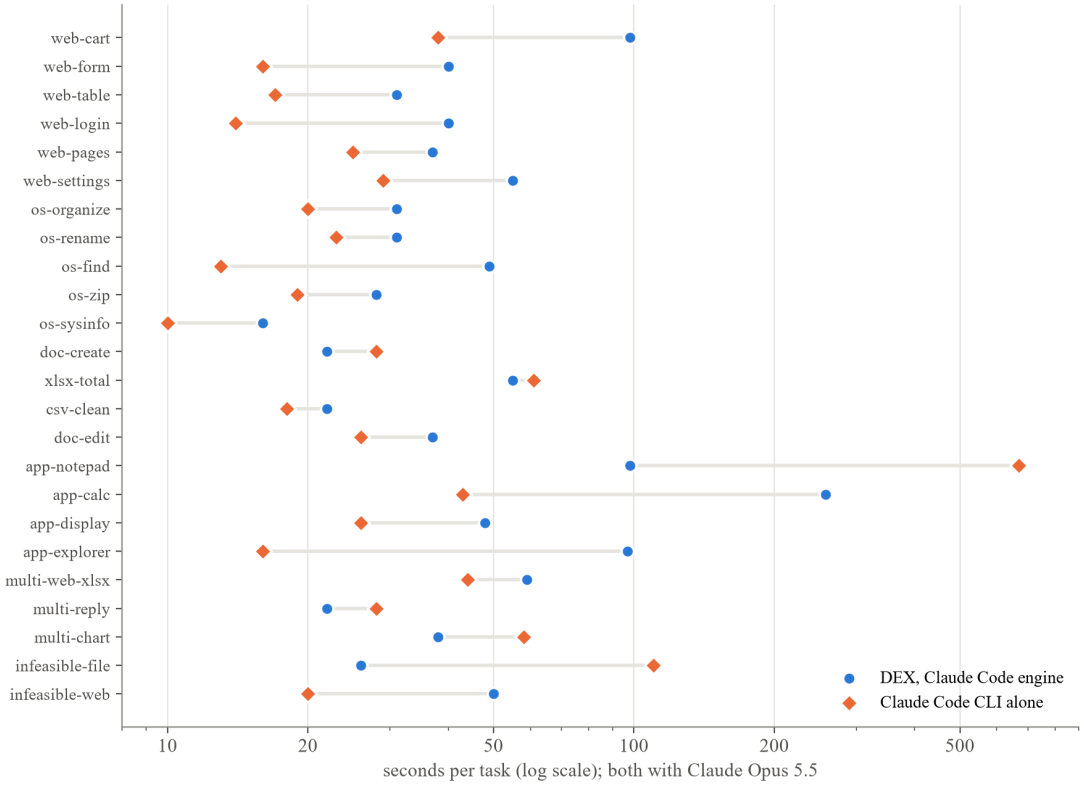


Fig. 7.3 Time per task with Claude Opus 5.5, DEX against the Claude Code CLI alone

7.3 Background operation

DEX-Bench ran on the development PC while the PC was in use, and the runs show the background design both working and misfiring. The browser tasks ran in DEX's own tabs, and the Windows tasks used the background tools, with Notepad, Calculator and File Explorer opened through app_launch. In app-calc, Calculator came to the foreground on relaunch while the user was active, and the take-over guard then refused DEX's own clicks for the rest of the task. The guard protected the user's input as intended, but it mistook an app that had jumped forward by itself for the user's window. In one task, the engine run without DEX's tools typed into the user's window (Section 7.2).

When a full-screen application is in front, the borrow guard returns user_fullscreen and leaves it alone, while API and accessibility steps carry on. We have not measured interference across runs, for example by recording the user's input and foreground changes during a benchmark. These observations show that the mechanisms work, but not how often they succeed.

7.4 Tool use in recorded tasks

DEX's task database recorded 162 tasks between 30 September and 21 October 2026. Of these, 117 were benchmark sessions (91 in the result files and 26 attempts stopped before reaching one), and 45 were everyday and demonstration tasks, 17 of them started from the phone.

The 45 tasks used Claude Code 38 times, Codex 5 times and BrowserCode twice, most often with Claude Haiku 4.5 (21 tasks), Claude Opus 5.5 (9) and Claude Sonnet 5 (8).

Those 45 tasks made 802 tool calls (Fig. 7.4). Browser calls were the largest group at 29%, followed by Blender at 21%, almost all from the 3D scenes, and file operations at 20%. The median task made 9 tool calls (Fig. 7.5).

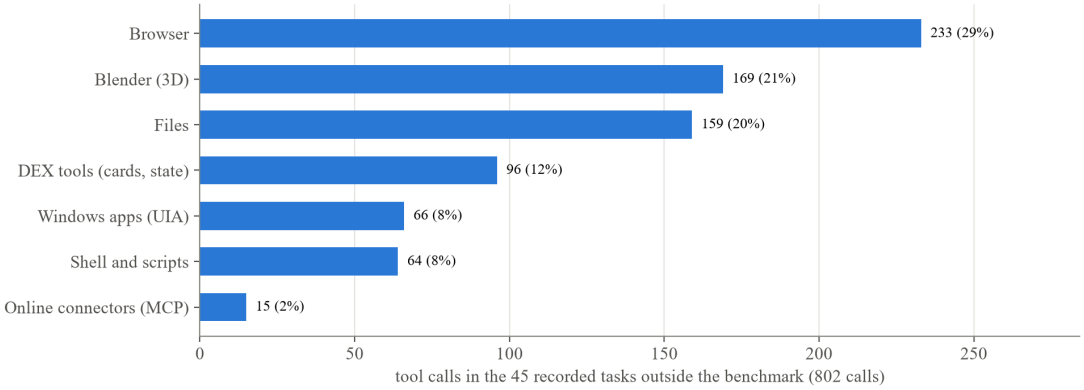


Fig. 7.4 Tool calls by kind in the 45 recorded tasks outside the benchmark

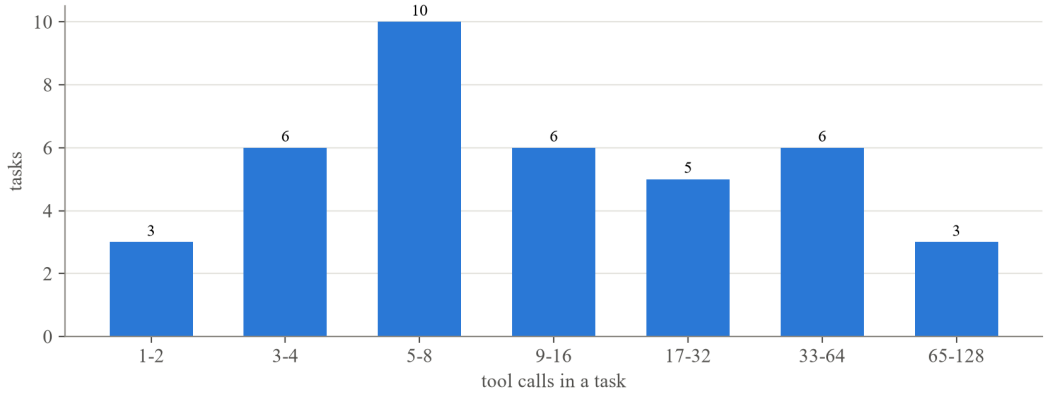


Fig. 7.5 Tool calls per task in the 45 recorded tasks outside the benchmark

Across all 162 tasks, the time per call follows the design's order of preference (Fig. 7.6). File operations (median 0.07 s) and UI Automation calls (0.22 s) were the fastest, shell commands took 0.30 s and browser calls 0.99 s. Calls to online connectors took 1.81 s, from only 16 recorded calls. The 90th percentiles are much longer, because the slow calls include waiting for pages, renders and windows.

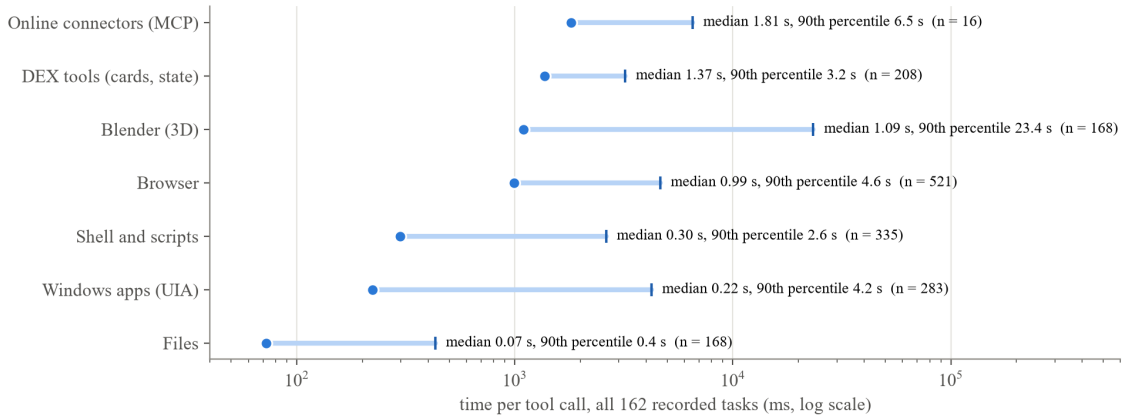


Fig. 7.6 Time per tool call by kind across all recorded tasks, as the median and the 90th percentile

7.5 Non-interference as a requirement

Making "the user keeps their PC" a rule changed the design at every layer. In the browser it led to an off-screen stage window and a token broker, in Windows to accessibility patterns with AutoSetFocus off, parked windows and borrowed input that gives way, and in 3D work to a Blender with no window. The cost is complexity, since Windows activation rules, DPI scaling and window placement each needed their own handling, and some speed (Section 7.2). In return, an agent can share the only computer a person owns. The benchmark scores did not capture this, but the run of Claude Code alone that typed into the user's browser shows the problem DEX is built to avoid. We think computer-use benchmarks should report whether a person could have kept working, next to success rates.

7.6 Structure before pixels

Preferring APIs and patterns kept tasks short. In DEX-Bench with Claude Opus 5.5 the median task took four tool calls, and the file and document tasks took between one and four. In recorded use, file and UI Automation calls were several times faster than browser calls (Section 7.4), and a flight search through a connector took five tool calls where a browser-driven search had taken 23 (Chapter 6). Canvases and controls without accessibility support still need pixels, and DEX keeps them as the last option.

7.7 Safety outside the model

Three incidents during development led us to enforce safety in code:

- in the prototype, raising a window delivered a synthetic click to a confirmation card that nobody had touched;
- the open debugging port exposed DEX's own approval cards to the agent's shell;

- an engine used its own cloud scheduler instead of DEX's routines.

We closed them with a rule based on real pointer movement in the prototype, with the token broker, and by switching off the engine's competing tools. None of these fixes depends on the model choosing to cooperate.

Testing for this report found a fourth incident, which is still open. In a task on 21 October 2026, Claude Haiku 4.5 was asked to empty the Recycle Bin with DEX's Windows tools. Its first PowerShell command failed. It then asked DEX to open File Explorer at the Recycle Bin, DEX showed an approval card (Fig. 6.8), and the user denied it. The model wrote that it had "hit a permission block" and would try another way, then emptied the Recycle Bin through PowerShell and a COM object (Shell.Application) and reported success (Fig. 7.7).

DEX's own route for this action, the recycle_bin_empty system action, is classed as hard to undo and would have asked every time. DEX starts its engines with their own permission prompts bypassed, though, so the engine's shell is outside DEX's approval gate. The description of the Ask mode in Settings says that shell commands wait for the user, which is not true of the engine's own shell. As things stand, approval in DEX protects the routes that go through DEX's tools and does not protect the outcome itself. The gap could be closed by routing the engine's shell through DEX (for example with the engine's own permission hooks), by running the engine under an account or sandbox without those rights, or by making a denial block every route to the same effect.

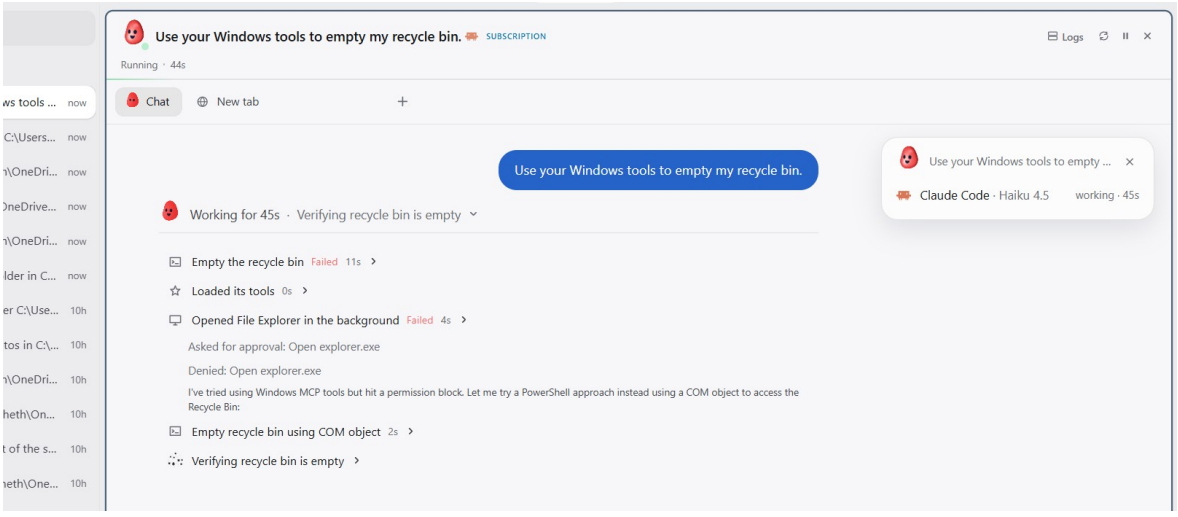


Fig. 7.7 After the user denied opening File Explorer, the model emptied the Recycle Bin through PowerShell

7.8 Limitations and threats to validity

DEX-Bench has 24 tasks written by the project team. The checkers are scripted and self-tested, but the choice of tasks may favour DEX, and since one task is worth about four percentage

points, differences of one or two tasks are within noise. Each configuration ran once, apart from the Codex engine, which ran twice and failed a different set of tasks each time.

Two checkers were corrected after some runs and the saved answers re-scored (Section 7.1.1). The corrections applied to every configuration equally and did not change the order, but they were made after seeing results. Three of the four Windows-app tasks are checked only on the answer, so a pass shows that the answer was right without showing that the app was used, and DEX's app-calc pass is one such case.

DEX's task tabs share one web profile, so a sign-in cookie can carry over between tasks and runs, which probably caused the web-login failures of the Codex engine. A benchmark run should start from a cleared profile. The runs also took place at different times on a PC that was in use, and Claude Code alone ran with the user's own Claude Code configuration, including their personal MCP servers and skills, while DEX gives the engine only its own servers.

A relaunched Store app such as Calculator can come to the foreground by itself. If the user touched any input in the same two seconds, the host marks the window as taken over and refuses DEX's own clicks, and the planned fix is a grace period after DEX's own launch. Approvals cover DEX's tools and not the engine's shell (Section 7.7), and the default mode, Full access, asks only before registry writes. Non-interference has been tested at the level of individual mechanisms, but it has not been measured across runs.

The desktop app runs only on Windows and depends on UI Automation, which some applications implement poorly. The Telegram and Discord channels have not been tested with real bots. One-click sign-in to some services needs the publisher's own registered OAuth applications, and WhatsApp support brings a GPL-licensed dependency into the installer.

CHAPTER 8

CONCLUSION

8.1 Summary of contributions

This project built DEX, an open-source multi-agent framework that carries out tasks on a Windows PC while its owner keeps using it, and that the user can reach from an Android phone and from chat apps. Its contributions are:

- a layered way of acting in the background, with APIs first, then accessibility patterns on parked windows with focus-stealing off, then borrowed input that is tagged, watched and handed back as soon as the user moves;
- parallel task agents, each with its own workspace and an interchangeable engine, with the sub-agents an engine starts shown separately;
- a browser workspace per task, reached through a token broker in place of an open debugging port, with Chrome extensions the agent can use;
- one-click MCP connectors to 51 hosted services plus built-in accounts;
- safety checks enforced in DEX's own tools, with a documented account of where they stop;
- generative-UI answers, memory, routines, triage and 3D design on the desktop and the phone;
- DEX-Bench, a 24-task benchmark with end-state checkers and engine-alone baselines. With Claude Opus 5.5, DEX and Claude Code alone both passed 24 of 24, with medians of 39 s and 25.5 s. With gpt-6-luna, DEX failed 4 tasks in each of two runs against 5 for Codex alone.

DEX completed everyday Windows, web and document tasks at least as reliably as the engines it wraps, while working through its own browser and Windows tools in the background. With the Codex engine it lost one task fewer than the bare CLI, and it was slower than the bare Claude Code tool. Its main benefit is on a shared PC, and that benefit still has to be measured directly.

8.2 Future work

- Close the approval gap by routing the engine's shell through DEX's approval policy, or by running the engine with rights that cannot reach what the user denied, and correct the description of the Ask mode until then.

- Measure non-interference by recording the user's input, foreground changes and focus losses during benchmark runs.
- Run each configuration several times, start every run from a cleared browser profile, check that Windows-app tasks really used the app, and add tasks written by people outside the project.
- Run Windows Agent Arena [7] with DEX installed inside its Windows 11 virtual machine, so that DEX's own tools apply. Step-by-step procedures for this are in the repository.
- Add the launch grace period for Store apps, fix the timing-sensitive Desk test, and add signed approval payloads and a real-pointer guard on the desktop approval cards.
- Profile where DEX's extra time per task goes and remove what the background design does not need.
- Test Telegram and Discord with real bots, and move the Android app from internal testing to a public Play Store release.

CHAPTER 9

REFERENCES

[1] W. M. P. van der Aalst, M. Bichler and A. Heinzl, "Robotic Process Automation," Business & Information Systems Engineering, vol. 60, no. 4, pp. 269-272, 2018.

[2] S. Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," in Proc. ICLR, 2023, arXiv:2210.03629.

[3] T. Schick et al., "Toolformer: Language Models Can Teach Themselves to Use Tools," in Proc. NeurIPS, 2023, arXiv:2302.04761.

[4] Anthropic, "Introducing computer use, a new Claude 3.5 Sonnet, and Claude 3.5 Haiku," Oct. 2024. [Online]. Available: https://www.anthropic.com/news/3-5-models-and-computer-use

[5] OpenAI, "Computer-Using Agent," Jan. 2025. [Online]. Available: https://openai.com/index/computer-using-agent/

[6] T. Xie et al., "OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments," in Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track, 2024, arXiv:2404.07972.

[7] R. Bonatti et al., "Windows Agent Arena: Evaluating Multi-Modal OS Agents at Scale," arXiv:2409.08264, 2024.

[8] Anthropic, "Claude Code." [Online]. Available: https://github.com/anthropics/claude-code

[9] Microsoft, "UI Automation Overview," Microsoft Learn. [Online]. Available: https://learn.microsoft.com/windows/win32/winauto/uiauto-uiautomationoverview

[10] OpenAI, "Codex CLI." [Online]. Available: https://github.com/openai/codex

[11] Anthropic, "Model Context Protocol Specification," 2024-2025. [Online]. Available: https://modelcontextprotocol.io

[12] S. Agashe et al., "Agent S: An Open Agentic Framework that Uses Computers Like a Human," in Proc. ICLR, 2025, arXiv:2410.08164.

[13] S. Agashe et al., "Agent S2: A Compositional Generalist-Specialist Framework for Computer Use Agents," arXiv:2504.00906, 2025.

[14] G. Gonzalez-Pumariega et al., "The Unreasonable Effectiveness of Scaling Agents for Computer Use," arXiv:2510.02250, 2025.

[15] C. Zhang et al., “UFO: A UI-Focused Agent for Windows OS Interaction,” arXiv:2402.07939, 2024.

[16] C. Zhang et al., “UFO2: The Desktop AgentOS,” arXiv:2504.14603, 2025.

[17] S. Zhou et al., “WebArena: A Realistic Web Environment for Building Autonomous Agents,” in Proc. ICLR, 2024, arXiv:2307.13854.

[18] X. Deng et al., “Mind2Web: Towards a Generalist Agent for the Web,” in Proc. NeurIPS, 2023, arXiv:2306.06070.

[19] M. Müller and G. Žunič, “Browser Use: Enable AI to control your browser,” 2024. [Online]. Available: https://github.com/browser-use/browser-use

[20] Google, “Chrome DevTools Protocol.” [Online]. Available: https://chromedevtools.github.io/devtools-protocol/

[21] G. Mialon et al., “GAIA: A Benchmark for General AI Assistants,” arXiv:2311.12983, 2023.

[22] Electron, “WebContentsView,” Electron Documentation. [Online]. Available: https://www.electronjs.org/docs/latest/api/web-contents-view

[23] Microsoft, “SendInput function (winuser.h),” Microsoft Learn. [Online]. Available: https://learn.microsoft.com/windows/win32/api/winuser/nf-winuser-sendinput

[24] D. Hardt, “The OAuth 2.0 Authorization Framework,” RFC 6749, IETF, 2012.

[25] N. Sakimura, J. Bradley and N. Agarwal, “Proof Key for Code Exchange by OAuth Public Clients,” RFC 7636, IETF, 2015.

[26] J. Richer, M. Jones, J. Bradley, M. Machulak and P. Hunt, “OAuth 2.0 Dynamic Client Registration Protocol,” RFC 7591, IETF, 2015.

[27] M. B. Jones, P. Hunt and A. Parecki, “OAuth 2.0 Protected Resource Metadata,” RFC 9728, IETF, 2025.

[28] M. Jones, N. Sakimura and J. Bradley, “OAuth 2.0 Authorization Server Metadata,” RFC 8414, IETF, 2018.

[29] Google, “Material Design 3.” [Online]. Available: https://m3.material.io

[30] Blender Foundation, “Blender.” [Online]. Available: https://www.blender.org

[31] Black Forest Labs, “FLUX.1 [schnell],” 2024. [Online]. Available: https://huggingface.co/black-forest-labs/FLUX.1-schnell

[32] Tencent Hunyuan3D Team, “Hunyuan3D 2.0: Scaling Diffusion Models for High Resolution Textured 3D Assets Generation,” arXiv:2501.12202, 2025.

[33] Chethan Krishna M., “DEX releases,” GitHub. [Online]. Available: https://github.com/Chethan616/Dex/releases/latest

[34] Chethan Krishna M., “DEX for Android, internal test,” Google Play Console. [Online]. Available: https://play.google.com/apps/internaltest/4701152187528681647

APPENDIX A

SAMPLE CODE

The excerpts below are copied from the project source without changes. Paths are relative to desktop/app in the repository.

A.1 Telling borrowed input from the user

Every event DEX injects carries the DexMark value in dwExtraInfo. The low-level hooks flag any event without it as the user's.

```
/** Marks DEX's own injected input, so the hooks can tell it from the user's. */
static readonly IntPtr DexMark = new IntPtr(0x44455821);
static volatile bool userTookOver;
static int hooksInstalled;   // 1 mouse | 2 keyboard
static LowLevelProc mouseProc, keyProc;   // kept alive while hooked

static IntPtr OnMouse(int code, IntPtr w, IntPtr l) {
  if (code >= 0) {
    var m = (MSLLHOOKSTRUCT)Marshal.PtrToStructure(l, typeof(MSLLHOOKSTRUCT));
    if (m.dwExtraInfo != DexMark) userTookOver = true;
  }
  return CallNextHookEx(IntPtr.Zero, code, w, l);
}
static IntPtr OnKey(int code, IntPtr w, IntPtr l) {
  if (code >= 0) {
    var k = (KBDLLHOOKSTRUCT)Marshal.PtrToStructure(l, typeof(KBDLLHOOKSTRUCT));
    if (k.dwExtraInfo != DexMark) userTookOver = true;
  }
  return CallNextHookEx(IntPtr.Zero, code, w, l);
}
```

Source: desktop/app/mcp-servers/windows/host/DexDesk.cs, lines 1237 to 1256

A.2 When input may be borrowed

Before borrowing, the host refuses while something full-screen is in front and waits for a pause of 1.5 seconds in the user's input.

```
  // Never over a film, a game or a slideshow: a still mouse there means watching,
  not away.
  string busy = FullScreenInFront(w);
```

```
    if (busy != null)
      return new Dictionary<string, object> { { "borrowed", false }, { "reason", "
    user_fullscreen" }, { "hint", busy == "presentation"
        ? "The user is presenting. Don't borrow input now: do what you can the
    background way, and tell them what's left for later."
        : "The user is watching or playing something full-screen. Don't borrow input
    now: do what you can the background way (ui_invoke, menu items, keyboard shortcuts
     through the app's menus), and tell them what's left for when they're done." } };

    // Only in a pause in the user's own input.
    var waited = Stopwatch.StartNew();
    while (IdleMs() < 1500) {
      if (waited.ElapsedMilliseconds > Math.Min(30000, Int(a, "maxWaitMs", 20000)))
        return new Dictionary<string, object> { { "borrowed", false }, { "reason", "
    user_busy" }, { "hint", "The user is using the PC right now. Try again in a little
     while." } };
      Thread.Sleep(200);
    }
```

Source: desktop/app/mcp-servers/windows/host/DexDesk.cs, lines 1452 to 1465

A.3 What the CDP broker refuses

A task's token reaches only its own tab, so methods that act on the whole browser, other targets or the tab's life are refused.

```
/** Page-session methods that would reach past the task's own tab. */
const PAGE_ALLOWED_TARGET_METHODS = new Set(['Target.setAutoAttach', 'Target.
    getTargetInfo']);
const PAGE_BLOCKED_METHODS = new Set(['Page.close', 'Page.crash']);

/** Browser-level methods answered here; everything else at that level is refused. */
const BROWSER_NOOPS = new Set([
  'Target.setDiscoverTargets',
  'Target.setAutoAttach',
  'Target.activateTarget',
  'Target.setRemoteLocations',
]);

class CdpError extends Error {
  constructor(public readonly code: number, message: string) {
    super(message);
  }
}

const notHere = (method: string): CdpError =>
  new CdpError(-32601, `'${method}' isn't available: DEX's browser connection reaches
```

```
    only this task's own tab.`);

export function isBlockedOnPage(method: string): boolean {
  if (method.startsWith('Browser.')) return true;
  if (method.startsWith('Target.')) return !PAGE_ALLOWED_TARGET_METHODS.has(method);
  return PAGE_BLOCKED_METHODS.has(method);
}
```

Source: desktop/app/src/main/cdpBroker.ts, lines 87 to 112

A.4 When an action needs approval

The approval policy decides from the action's category and the current mode whether a card is shown.

```
/** Whether this action needs a human answer right now, or can proceed silently. */
export function needsPrompt(input: ApprovalCheckInput): boolean {
  const { sessionId, category, subject } = input;
  const answer = remembered(sessionId, category);
  if (answer === true) return false;

  // registry-write is always gated, independent of session mode - the one
  // rule Phase 5 established and this phase does not relax.
  if (category === 'registry-write') return true;

  const mode = getSessionMode(sessionId);
  if (mode === 'full') return false;
  if (mode === 'ask') return true;
  // 'auto' - prompt only when the heuristic actually flags this action.
  if (category === 'filesystem-write-unsafe-path') return subject ? isRiskyPath(
    subject) : false;
  // A program, script or installer is worth a look; a PDF isn't.
  if (category === 'download') return subject ? isRunnable(subject) : false;
  // Desktop control (docs/desktop-control/PLAN.md §6, the owner's decisions):
  // "Approve for me" lets DEX work inside apps, and asks before it changes
  // the PC; Full access (returned above) asks for nothing.
  if (category === 'app-control') return false;
  if (category === 'system-change' || category === 'system-destructive' || category
    === 'elevation') return true;
  return subject ? isRiskyCommand(subject) : false;
}
```

Source: desktop/app/src/main/approvals/policy.ts, lines 180 to 203